FSD-2 LAB MANUAL

A report submitted in partial fulfilment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

INDEX

S.No.	Topic	Program Title	Page No.
1	Node.js	a. Create a web server to show JavaScript code execution workflow. b. Transfer data over HTTP using http module. c. Create src.txt file containing (HTML, CSS, JavaScript, TypeScript, MongoDB, ExpressJS, ReactJS, NodeJS). d. Parse a URL using the URL module. e. Create a user-defined module and demonstrate modularization in Node.js.	1
2	TypeScript	a. Program on simple and special types. b. Function parameters and return types. c. Arrow functions with optional, default, and REST parameters. d. Class with constructor, properties, methods, and access specifiers. e. Working with namespaces and modules. f. Generics with variables, functions, and constraints.	8
3	Augmented Programs 1	a. Apply 2D and 3D transformations in CSS. b. Design a web page using HTML5 and CSS3 features. c. Create a To-Do List using JavaScript.	16
4	ExpressJS – Routing, HTTP Methods, Middleware	a. Define routes, handle route & query parameters, and URL building. b. Accept, retrieve, and delete data using HTTP methods. c. Demonstrate working of middleware.	28
5	ExpressJS – Templating and Form Data	a. Use a templating engine. b. Work with form data in ExpressJS.	33
6	ExpressJS – Cookies, Sessions, Authentication	a. Session management using cookies and sessions. b. User authentication.	37
7	ExpressJS – Database & RESTful APIs	a. Connect MongoDB and perform CRUD operations. b. Develop a single-page application using RESTful APIs.	42
8	ReactJS – Render HTML, JSX, Components	a. Render HTML to a web page. b. Write markup using JSX. c. Create and nest components (function & class).	49
9	ReactJS – Props, States, Styles, Events	a. Work with props and states. b. Add CSS & Sass styling and display data. c. Respond to events.	52
10	ReactJS – Conditional Rendering, Lists, Forms	a. Conditional rendering. b. Rendering lists. c. Working with React forms and form fields.	60
11	ReactJS – Routing & Updating the Screen	a. Implement routing using React Router. b. Program for updating the screen.	71
12	ReactJS – Hooks & Data Sharing	a. Importance of React Hooks. b. Sharing data between components.	77
13	ReactJS Applications	Design a To-Do List application.	81
14	MongoDB – Installation & CRUD Operations	a. Install and configure MongoDB / MongoDB Atlas. b. Perform CRUD operations using insert(), find(), update(), and remove().	85
15	MongoDB – Databases, Collections & Records	a. Create and drop databases and collections. b. Work with records using find(), limit(), sort(), createIndex(), and aggregate().	89
16	Augmented Programs (Any 2)	a. Design a To-Do List using NodeJS and ExpressJS. b. Design a Quiz App using ReactJS. c. Complete MongoDB University Certification.	92

1. NodeJS

a. Write a program to show the workflow of JavaScript code executable by creating a web server in Node.js.

Server.js

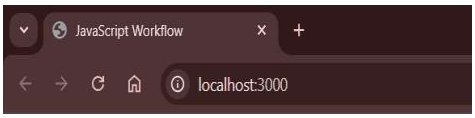
```
const http = require('http');
const fs = require('fs');
const path = require('path');
const PORT = 3000;
// Synchronous function
function compute(a, b) {
  return a + b;
}
// Create HTTP server
const server = http.createServer((req, res) => {
  console.log('Request received:', req.url);
  if (req.url === '/') {
    const filePath = path.join(__dirname, 'public', 'index.html');
    fs.readFile(filePath, (err, data) => {
      if (err) {
        res.writeHead(500, { 'Content-Type': 'text/plain' });
        res.end('Server Error');
        return;
      }
      res.writeHead(200, { 'Content-Type': 'text/html' });
      res.end(data);
    });
  }
  else if (req.url === '/workflow') {
    // Synchronous execution
    console.log('Start of synchronous code');
    const result = compute(10, 20);
    console.log('Computed result:', result);
    // Asynchronous execution
    console.log('Start of async task (setTimeout)');
    setTimeout(() => {
      console.log('Async task finished (after 2s)');
    }, 2000);
```

```
    // Immediate response
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({
      message: 'JavaScript workflow executed',
      result
    }));
    console.log(' End of request processing');
  }
  else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('404 Not Found');
  }
});
server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Index.html

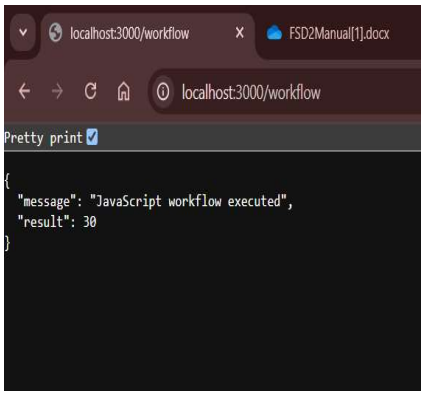
```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Workflow</title>
</head>
<body>
  <h1>Welcome to Node.js Server</h1>
  <p>Visit <a href="/workflow">/workflow</a> to see JavaScript execution
flow.</p>
</body>
</html>
```

Output



Welcome to Node.js Server

Visit /workflow to see JavaScript execution flow.



b. Write a program to transfer data over http protocol using http module.

Server.js

```
const http = require('http');
const server = http.createServer((req, res) => {
  if (req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ message: 'Hello from the server This is Akhil!' }));
  } else {
    res.writeHead(405, { 'Content-Type': 'text/plain' });
    res.end('Method Not Allowed');
  }
});
const PORT = 3000;
server.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

Client.js

```
const http = require('http');
const options = {
  hostname: 'localhost',
  port: 3000,
  path: '/',
  method: 'GET'
```

```
};
const req = http.request(options, (res) => {
  let data = '';
  res.on('data', (chunk) => {
    data += chunk;
  });
  res.on('end', () => {
    console.log('Response from server:', data);
  });
});
req.on('error', (err) => {
  console.error('Error:', err.message);
});
req.end();
```

Output

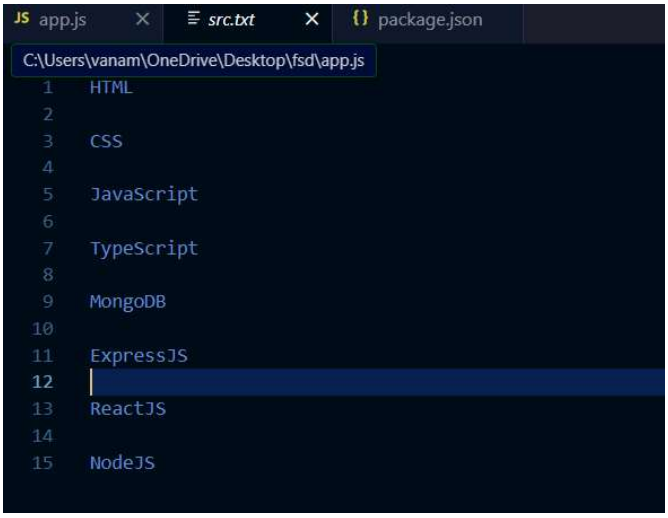


c. Create a text file src.txt and add the following content to it. (HTML, CSS, JavaScript, TypeScript, MongoDB, ExpressJS, ReactJS, NodeJS)

App.js

```
const fs = require('fs');
const content = `HTML
CSS
JavaScript
TypeScript
MongoDB
ExpressJS
ReactJS
NodeJS`;
fs.writeFile('src.txt', content, (err) => {
  if (err) {
    return console.error('Error writing file:', err);
  }
  console.log('src.txt created successfully with the content.');
});
```

output



d. Write a program to parse an URL using URL module

ParseUrl.js

```
const url = require('url');
const websiteUrl =
'https://www.example.com:8080/path/page.html?name=akhilCage=21#section2';
const parsedUrl = url.parse(websiteUrl, true);
console.log('Full URL:', parsedUrl.href);
console.log('Protocol:', parsedUrl.protocol);
console.log('Host:', parsedUrl.host);
console.log('Hostname:', parsedUrl.hostname);
console.log('Port:', parsedUrl.port);
console.log('Pathname:', parsedUrl.pathname);
console.log('Query String:', parsedUrl.query);
console.log('Fragment (Hash):', parsedUrl.hash);
```

Output



e. Write a program to create a user-defined module and show the workflow of Modularization of application using Node.**js**

MathUtil.js

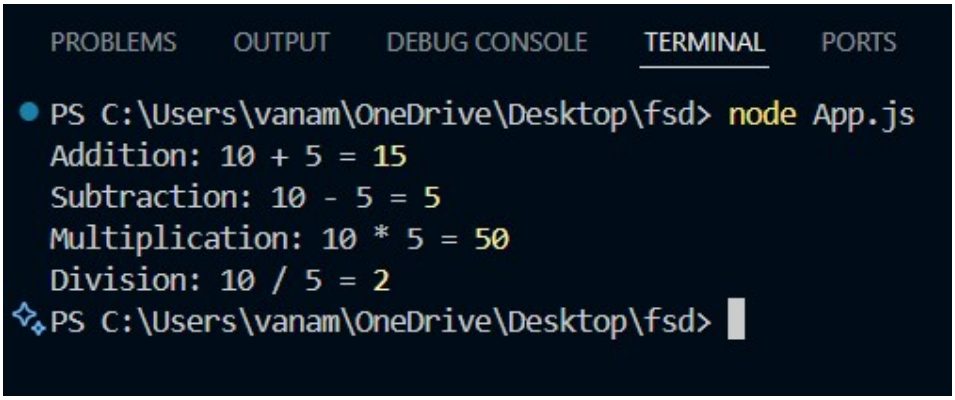
```
function add(a, b) {
  return a + b;
}
function subtract(a, b) {
  return a - b;
}
function multiply(a, b) {
  return a * b;
}
function divide(a, b) {
  if (b === 0) {
    return 'Cannot divide by zero!';
  }
  return a / b;
}
module.exports = {
  add,
```

```
  subtract,
  multiply,
  divide
};
```

App.js

```
const math = require('./mathUtil');
console.log('Addition: 10 + 5 =', math.add(10, 5));
console.log('Subtraction: 10 - 5 =', math.subtract(10, 5));
console.log('Multiplication: 10 * 5 =', math.multiply(10, 5));
console.log('Division: 10 / 5 =', math.divide(10, 5));
```

Output



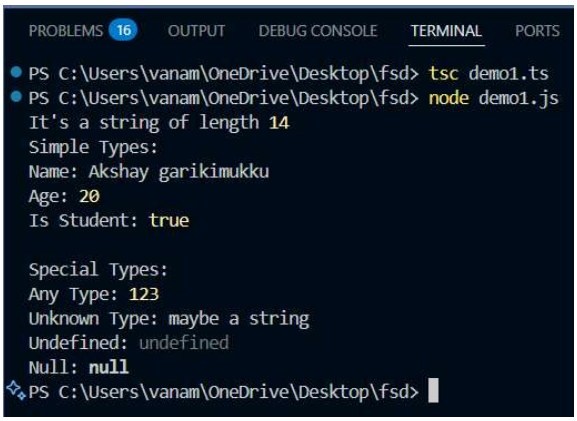
2. TypeScript

a. Write a program to understand simple and special types.

Demo.tsc

```
let age: number = 19;
let fullName: string = "Akhil Vanama";
let isStudent: boolean = true;
let anything: any = "Hello";
anything = 123;
let unknownValue: unknown = "maybe a string";
if (typeof unknownValue === "string") {
 console.log("It's a string of length", unknownValue.length);}
function logMessage(): void {
 console.log("This function returns nothing (void)");
}
function throwError(): never {
 throw new Error("This function never returns");
}
let u: undefined = undefined;
let n: null = null;
console.log("Simple Types:");
console.log("Name:", fullName);
console.log("Age:", age);
console.log("Is Student:", isStudent);
console.log("\nSpecial Types:");
console.log("Any Type:", anything);
console.log("Unknown Type:", unknownValue);
console.log("Undefined:", u);
console.log("Null:", n);
```

Output



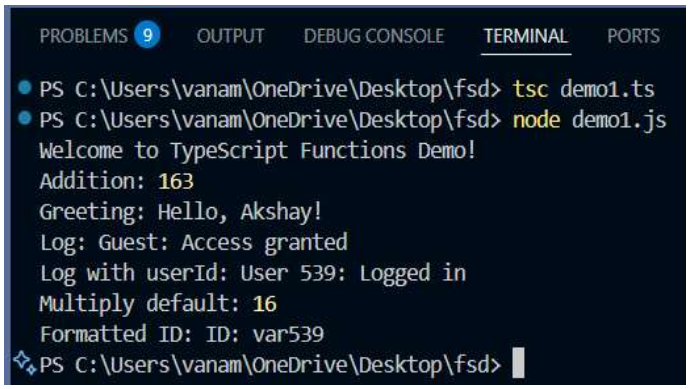
b. Write a program to understand function parameter and return types.

FunctionTypes.ts

```
function add(a: number, b: number): number {
 return a + b;
}
function greet(name: string): string {
 return `Hello, ${name}!`;
}
function log(message: string, userId?: number): string {
 return userId ? `User ${userId}: ${message}` : `Guest: ${message}`;
}
function multiply(x: number, y: number = 2): number {
 return x * y;
}
function printWelcome(): void {
 console.log("Welcome to TypeScript Functions Demo!");
}
function formatId(id: number | string): string {
 return `ID: ${id}`;
```

```
}
printWelcome();
console.log("Addition:", add(88, 75));
console.log("Greeting:", greet("Akhil"));
console.log("Log:", log("Access granted"));
console.log("Log with userId:", log("Logged in", 539));
console.log("Multiply default:", multiply(8));
console.log("Formatted ID:", formatId("var539"));
```

Output



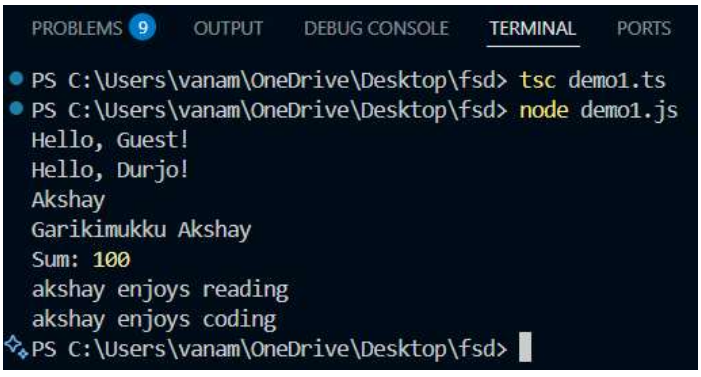
c. Write a program to show the importance with arrow functions. Use optional, default and REST parameters.

ArrowFunction.tsc

```
const greet = (name: string = "Guest"): string => {
 return `Hello, ${name}!`;
};
const getFullName = (firstName: string, lastName?: string): string => {
 return lastName ? `${firstName} ${lastName}` : firstName;
};
const sum = (...numbers: number[]): number => {
 return numbers.reduce((total, num) => total + num, 0);
};
```

```
const user = {
 name: "akhil",
 hobbies: ["reading", "coding"],
 showHobbies: function () {
 this.hobbies.forEach((hobby) => {
 console.log(`${this.name} enjoys ${hobby}`);
 });
 }
};
console.log(greet());
console.log(greet("Durjo"));
console.log(getFullName("Akhil"));
console.log(getFullName("Vanama", "Akhil"));
console.log("Sum:", sum(10, 20, 30, 40));
user.showHobbies();
```

Output



d. Write a program to understand the working of TypeScript with class, constructor, properties, methods and access specifiers.

EmployeDemo.ts

```
class Employee {
```

```
  public name: string;
  private salary: number;
  protected department: string;
  readonly id: number;
  constructor(id: number, name: string, salary: number, department: string) {
    this.id = id;
    this.name = name;
    this.salary = salary;
    this.department = department;
  }
  public getDetails(): string {
    return `ID: ${this.id}, Name: ${this.name}, Department: ${this.department}`;
  }
  private getSalary(): string {
    return `Salary: ₹${this.salary}`;
  }
  public showSalary(): void {
    console.log(this.getSalary());
  }
}
class Manager extends Employee {
  constructor(id: number, name: string, salary: number, department: string) {
    super(id, name, salary, department);
  }
  public showDepartment(): void {
    console.log(`Manager of ${this.department} department`);
  }
}
const emp1 = new Employee(101, "Akhil", 50000, "IT");
console.log(emp1.getDetails());
emp1.showSalary();
const mgr1 = new Manager(201, "Prabhas", 80000, "Design");
console.log(mgr1.getDetails());
mgr1.showDepartment();
```

output

```
PS C:\Users\vanam\OneDrive\Desktop\fsd> tsc demo1.ts
PS C:\Users\vanam\OneDrive\Desktop\fsd> node demo1.js
ID: 101, Name: Akshay, Department: cse
Salary: ₹50000
ID: 201, Name: Prabhas, Department: Design
Manager of Design department
PS C:\Users\vanam\OneDrive\Desktop\fsd>
```

...

e. Write a program to understand the working of namespaces and modules.

NamespaceDemo.ts

```
namespace MathUtils {
  export function add(a: number, b: number): number {
    return a + b;
  }
  export function multiply(a: number, b: number): number {
    return a * b;
  }
  export class Calculator {
    static square(x: number): number {
      return x * x; }
  }
}
console.log("Addition:", MathUtils.add(5, 3));
console.log("Multiplication:", MathUtils.multiply(4, 2));
console.log("Square:", MathUtils.Calculator.square(6));
```

Output

```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                powershell - e
PS E:\fsd2\TypeScript\e> tsc nameSpaceDemo.ts
PS E:\fsd2\TypeScript\e> node nameSpaceDemo.js
Addition: 8
Multiplication: 8
Square: 36
PS E:\fsd2\TypeScript\e>
```

MathModule.ts

```
export function subtract(a: number, b: number): number {
  return a - b;
}
export function divide(a: number, b: number): number {
  return b !== 0 ? a / b : NaN;
}
```

Main.ts

```
import { subtract, divide } from './mathModule';
console.log("Subtraction:", subtract(10, 3));
console.log("Division:", divide(10, 2));
```

Output

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                powershell - e
PS E:\fsd2\TypeScript\e> tsc main.ts mathModule.ts --module commonjs
PS E:\fsd2\TypeScript\e> node main.js
Subtraction: 7
Division: 5
PS E:\fsd2\TypeScript\e>
```

f. Write a program to understand generics with variables, functions and constraints.

GenericDemo.ts

```
type Box<T> = {
value: T;
};
```

```
let numberBox: Box<number> = { value: 42 };
let stringBox: Box<string> = { value: "Hello, TypeScript" };
console.log("Number Box:", numberBox.value);
console.log("String Box:", stringBox.value);
function identity<T>(arg: T): T {
 return arg;
}
console.log("Identity (number):", identity<number>(100));
console.log("Identity (string):", identity("Generic Function"));
function merge<T, U>(obj1: T, obj2: U): T C U {
 return { ...obj1, ...obj2 };
}
const mergedObj = merge({ name: "Akhil" }, { age: 21 });
console.log("Merged Object:", mergedObj);
interface HasLength {
 length: number;
}
function logLength<T extends HasLength>(item: T): void {
 console.log("Length is:", item.length);
}
logLength("Hello World");
logLength([1, 2, 3, 4]);
```

Output

```
PS C:\Users\vanam\OneDrive\Desktop\fsd> tsc demo1.ts
PS C:\Users\vanam\OneDrive\Desktop\fsd> node demo1.js
ID: 101, Name: Akshay, Department: cse
Salary: ₹50000
ID: 201, Name: Prabhas, Department: Design
Manager of Design department
PS C:\Users\vanam\OneDrive\Desktop\fsd>
```

3. Augmented Programs 1

a. Write a CSS program, to apply 2D and 3D transformations in a web page.

Index.html

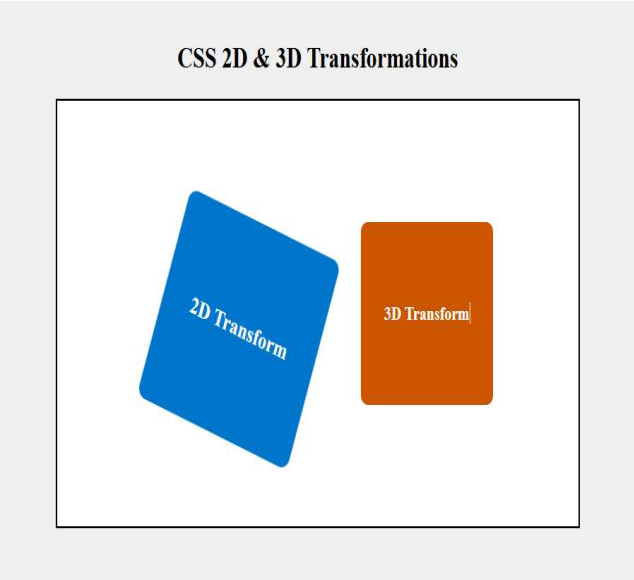
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>2D C 3D Transformations</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h2>CSS 2D C 3D Transformations</h2>
  <div class="container">
    <div class="box2d">2D Transform</div>
    <div class="box3d">3D Transform</div>
  </div>
</body>
</html>
```

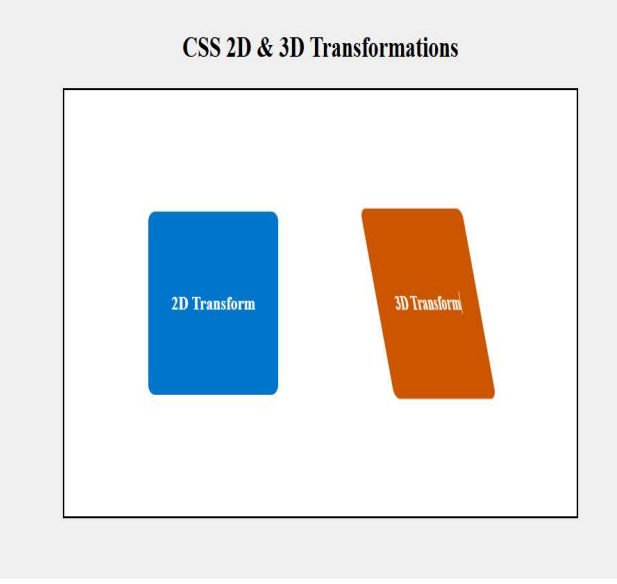
Style.css

```
body {
  background: #f0f0f0;
  margin: 0;
  height: 100vh;
  display: flex;
   flex-direction:  column;
  justify-content:   center;
  align-items: center;
}
.container {
  width: 85vh;
  height: 50vh;
  display: flex;
  justify-content: space-evenly;
  align-items: center;
  border: 2px solid black;
  background: white;
}
.box2d,
.box3d {
  width: 150px;
  height: 150px;
  text-align: center;
  line-height: 150px;
  color: white;
  font-weight: bold;
  border-radius: 8px;
  transition: transform 0.5s ease;
```

```
  cursor: pointer;
}
.box2d {
  background-color: #0077cc;
}
.box3d {
  background-color: #cc5500;
  transform-style: preserve-3d;
  perspective: 500px;
}
.box2d:hover {
  transform: rotate(20deg) scale(1.2) translateX(30px);
}
.box3d:hover {
  transform: rotateY(45deg) rotateX(20deg) scale(1.1);
}
```

Output





b. Design a web page with new features of HTML5 and CSS3

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="style.css">
  <title>Document</title>
</head>
<body>
   <header>
  <h1>Welcome to HTML5 C CSS3 Demo</h1>
  <nav>
   <a href="#">Home</a>
   <a href="#">Features</a>
   <a href="#">Contact</a>
  </nav>
 </header>
 <section>
  <article>
   <h2>HTML5 Features</h2>
   <p>HTML5 introduces semantic tags like <code>Clt;headerCgt;</code>,
<code>Clt;sectionCgt;</code>,
     <code>Clt;articleCgt;</code>, and <code>Clt;footerCgt;</code>, making pages
more structured.</p>
   <video controls>
```

```
      <source src="sample.mp4" type="video/mp4">
      Your browser does not support video.
    </video>
   </article>
   <article>
    <h2>CSS3  Features</h2>
    <p>CSS3 adds Flexbox, Grid, animations, transitions, shadows, and responsive design with media queries.</p>
    <audio controls>
     <source src="audio.mp3" type="audio/mpeg">
     Your browser does not support audio.
    </audio>
   </article>
  </section>
  <footer>
   <p>Ccopy; 2025 Modern Web Page Demo. All Rights Reserved.</p>
  </footer>
</body>
</html>
```

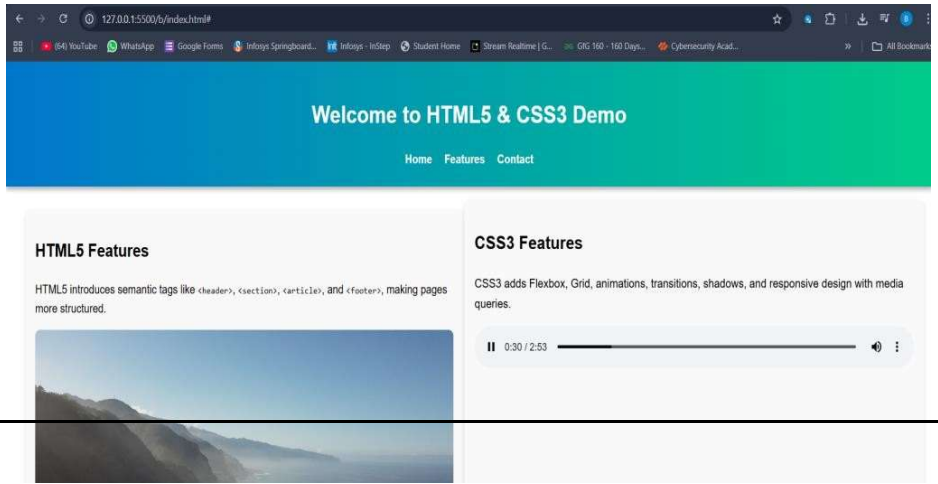
Style.css

```
  body {
  margin: 0;
    font-family: 'Arial', sans-serif;
    line-height: 1.6;
  }
  header {
    background: linear-gradient(90deg, #0077cc, #00cc88);
    color: white;
```

```
  text-align: center;
  padding: 1.5rem;
  box-shadow: 0 4px 6px rgba(0,0,0,0.3);
}
nav {
  display: flex;
  justify-content: center;
  gap: 20px;
  margin-top:  10px;
}
nav a {
  color: white;
  text-decoration: none;
  font-weight: bold;
  transition: color 0.3s;
}
nav a:hover {
  color: yellow;
}
section {
  padding: 2rem;
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 20px;
}
article {
  background: #f9f9f9;
  padding: 1rem;
```

```
  border-radius: 10px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.1);
  transition: transform 0.3s;
}
article:hover {
  transform: scale(1.05);
}
video, audio {
  width: 100%;
  border-radius: 8px;
}
footer {
  background: #222;
  color: #fff;
  text-align: center;
  padding: 1rem;
  margin-top: 20px;
}
@media (max-width: 768px) {
  section {
    grid-template-columns: 1fr;  }
}
```

Output



c. Design a to-do list application using JavaScript.

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>To-Do List App</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="todo-container">
    <h1>To-Do List</h1>
    <div class="input-group">
      <input type="text" id="taskInput" placeholder="Enter a new task" />
      <button onclick="addTask()">Add</button>
    </div>
    <ul id="taskList"></ul>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

Script.js

```
function addTask() {
  const input = document.getElementById("taskInput");
  const taskText = input.value.trim();
  if (taskText === "") {
    alert("Please enter a task!");
    return;
  }
  const list = document.getElementById("taskList");
  const li = document.createElement("li");
  li.addEventListener("click", function () {
    li.classList.toggle("completed");
  });
  li.textContent = taskText;
  const deleteBtn = document.createElement("button");
  deleteBtn.textContent = "✕";
  deleteBtn.addEventListener("click", function (e) {
    e.stopPropagation();
    list.removeChild(li);
  });
  li.appendChild(deleteBtn);
  list.appendChild(li);
  input.value = "";
}
```

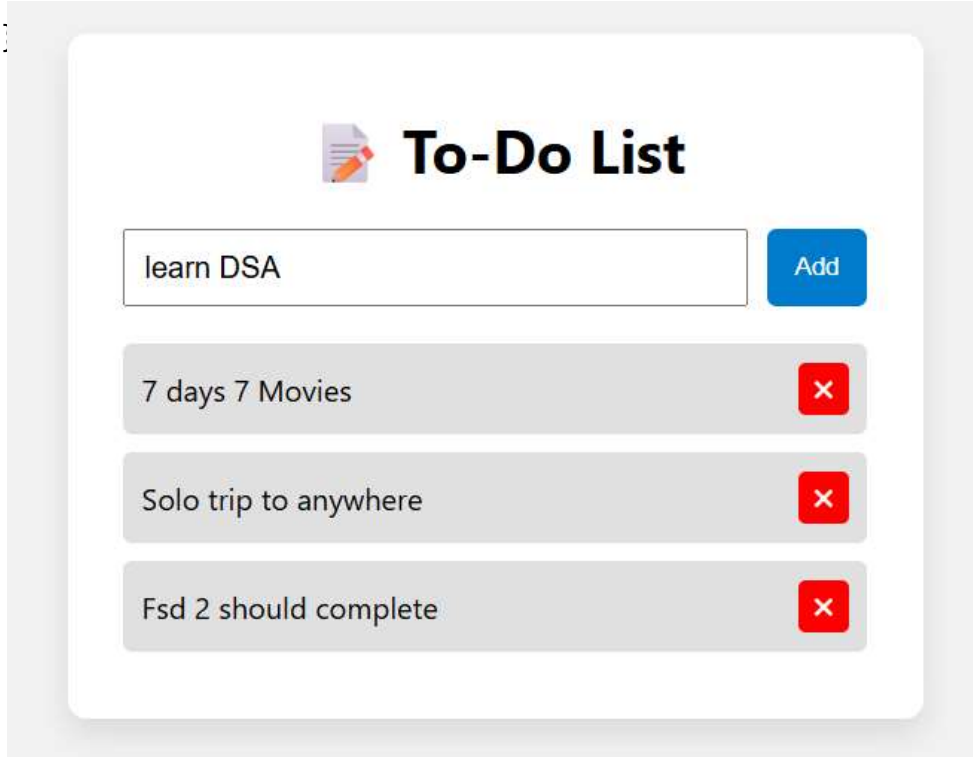
Style.css

```
body {
  font-family: 'Segoe UI', sans-serif;
```

```
  background: #f2f2f2;
  display: flex;
  justify-content: center;
  align-items: flex-start;
  padding-top: 50px;
}
.todo-container {
  background: #fff;
  padding: 20px 30px;
  border-radius: 10px;
  width: 100%;
  max-width: 400px;
  box-shadow: 0 8px 16px rgba(0, 0, 0, 0.1);
}
h1 {
  text-align: center;
}
.input-group {
  display: flex;
  gap: 10px;
}
input[type="text"] {
  flex: 1;
  padding: 10px;
  font-size: 16px;
}
button {
  padding: 10px 15px;
```

```
  background-color: #007acc;
  color: white;
  border: none;
  cursor: pointer;
  border-radius: 5px;
}
button:hover {
  background-color: #005f99;
}
ul {
  list-style-type: none;
  padding: 0;
  margin-top: 20px;
}
li {
  padding: 10px;
  margin-bottom: 10px;
  background: #e0e0e0;
  display: flex;
  justify-content: space-between;
  align-items: center;
  border-radius: 5px;
}
li.completed {
  text-decoration: line-through;
  background-color: #c8e6c9;
}
li button {
```

```
  background: red;
  color: white;
  border: none;
  padding: 5px 8px;
  border-radius: 4px;
  cursor: pointer;
}
li button:hover {
  background: darkred;
}
```



4. ExpressJs

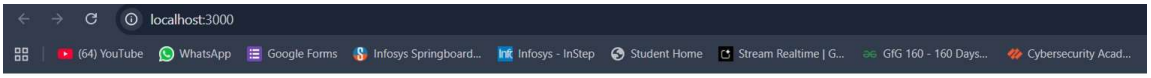
a. Write a program to define a route, Handling Routes, Route Parameters, Query Parameters and URL building.

App.js

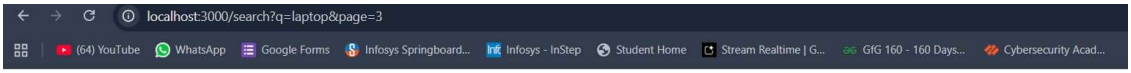
```
const express = require('express');
const app = express();
const PORT = 3000;
app.get('/', (req, res) => {
  res.send('Welcome to the Home Page!');
});
app.get('/user/:username', (req, res) => {
  const username = req.params.username;
  res.send(`Hello, ${username}! This is your profile page.`);
});
app.get('/product/:category/:id', (req, res) => {
  const { category, id } = req.params;
  res.send(`Viewing product ID: ${id} in category: ${category}`);
});
app.get('/search', (req, res) => {
  const { q, page = 1 } = req.query;
  if (q) {
    res.send(`Search results for: "${q}" - Page ${page}`);
  } else {
    res.send('Please enter a search query using "?q=your_query"');
  }
});
app.get('/redirect-to-user', (req, res) => {
  const username = 'akshay_garikimukku';
  const url = `/user/${username}`;
  res.redirect(url);
});
app.get('/build-url', (req, res) => {
  const username = 'babloo';
  const query = '?page=2Csort=desc';
  const fullUrl = `/user/${username}${query}`;
  res.send(`Built URL: <a href="${fullUrl}">${fullUrl}</a>`);
});
app.listen(PORT, () => {
```

```
  console.log(`Server running at http://localhost:${PORT}`);
})
```

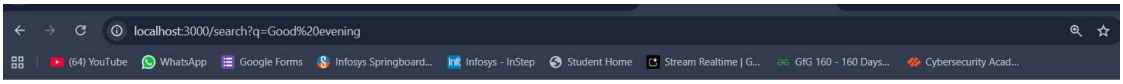
Output



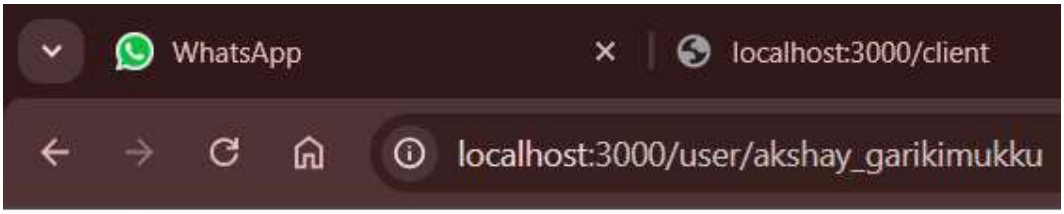
Welcome to the Home Page!



Search results for: "laptop" - Page 3



Search results for: "Good evening" - Page 1



Hello, akshay_garikimukku! This is your profile page.

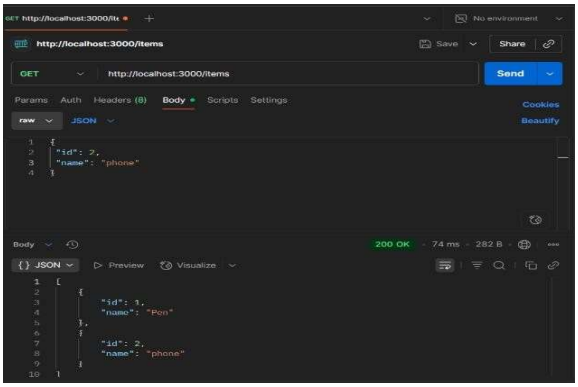
b. Write a program to accept data, retrieve data and delete a specified resource using http methods.

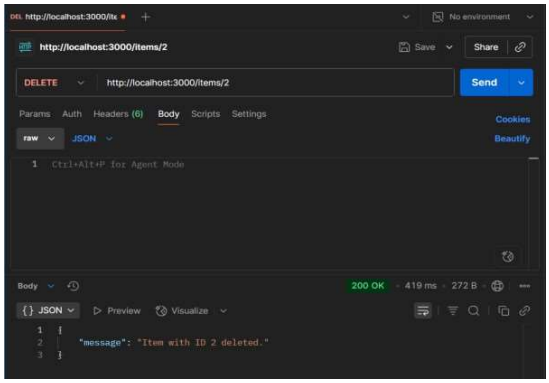
Server.js

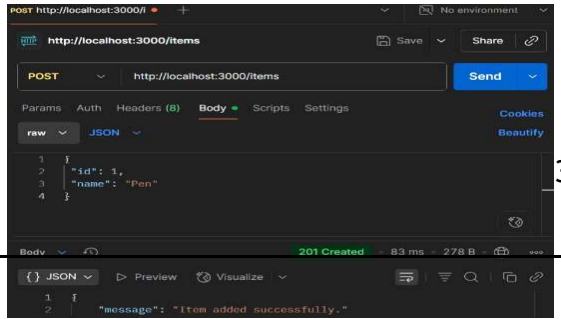
```
const express = require('express');
const app = express();
const PORT = 3000;
app.get('/', (req, res) => {
  res.send('Welcome to the Express server!');
});
app.use(express.json());
let items = [];
app.get('/items', (req, res) => {
```

```
  res.json(items);
});
app.post('/items', (req, res) => {
  const { id, name } = req.body;
  if (!id || !name) {
    return res.status(400).json({ message: "ID and Name are required." });
  }
  const exists = items.find(item => item.id === id);
  if (exists) {
    return res.status(409).json({ message: "Item with this ID already exists." });
  }
  items.push({ id, name });
  res.status(201).json({ message: "Item added successfully." });
});
app.delete('/items/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = items.findIndex(item => item.id === id);
  if (index === -1) {
    return res.status(404).json({ message: "Item not found." });
  }
  items.splice(index, 1);
  res.json({ message: `Item with ID ${id} deleted.` });
});
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Output







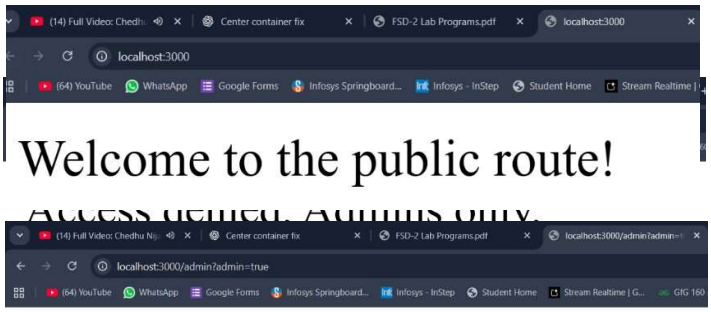
c. Write a program to show the working of middleware

Middleware.js

```
const express = require('express');
const app = express();
const PORT = 3000;
app.use((req, res, next) => {
  console.log(`[LOG] ${req.method} - ${req.url}`);
  next();
});
const checkAdmin = (req, res, next) => {
  const isAdmin = req.query.admin;
  if (isAdmin === 'true') {
    console.log('Admin access granted');
    next();
  } else {
    res.status(403).send('Access denied. Admins only.');
  }
};
app.get('/', (req, res) => {
  res.send('Welcome to the public route!');
});
app.get('/admin', checkAdmin, (req, res) => {
  res.send('Welcome to the admin panel.');
});
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
```

});

output


(14) Full Video: Chedh
Center container fix
FSD-2 Lab Programs.pdf
localhost:3000
localhost:3000
(64) YouTube
WhatsApp
Google Forms
Infosys Springboard...
Infosys - InStep
Student Home
Stream Realtime |
Welcome to the public route!
Access denied. Admins only.

(14) Full Video: Chedhu Nij
Center container fix
FSD-2 Lab Programs.pdf
localhost:3000/admin?admin=
localhost:3000/admin?admin=true
(64) YouTube
WhatsApp
Google Forms
Infosys Springboard...
Infosys - InStep
Student Home
Stream Realtime | G...
GTG 160
Welcome to the admin panel.

5. ExpressJS – Templating, Form Data

a. Write a program using templating engine.

App.js

```
const express = require('express');
const app = express();
const PORT = 3000;
app.set('view engine', 'ejs');
app.get('/', (req, res) => {
  const title = 'Welcome Page';
  res.render('home', { title });
});
app.get('/user/:name', (req, res) => {
  const name = req.params.name;
  res.render('user', { name });
});
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

Views/home.ejs

```
<!DOCTYPE html>
<html>
<head>
  <title>
    <%= title %>
  </title>
</head>
<body>
```

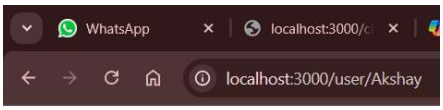
```
  <h1>
    <%= title %>
  </h1>
  <p>This page is rendered using the EJS template engine.</p>
  <a href="/user/Akshay">Go to User Page</a>
</body>
</html>
```

Views/user.ejs

```
<!DOCTYPE html>
<html>
<head>
  <title>User Page</title>
</head>
<body>
  <h1>Hello, <%= name %>!</h1>
  <p>This is a dynamic user profile page.</p>
  <a href="/">Back to Home</a>
</body>
</html>
```



output



Hello, Akshay!

This is a dynamic user profile page.

Back to Home

b. Write a program to work with form data.

App.js

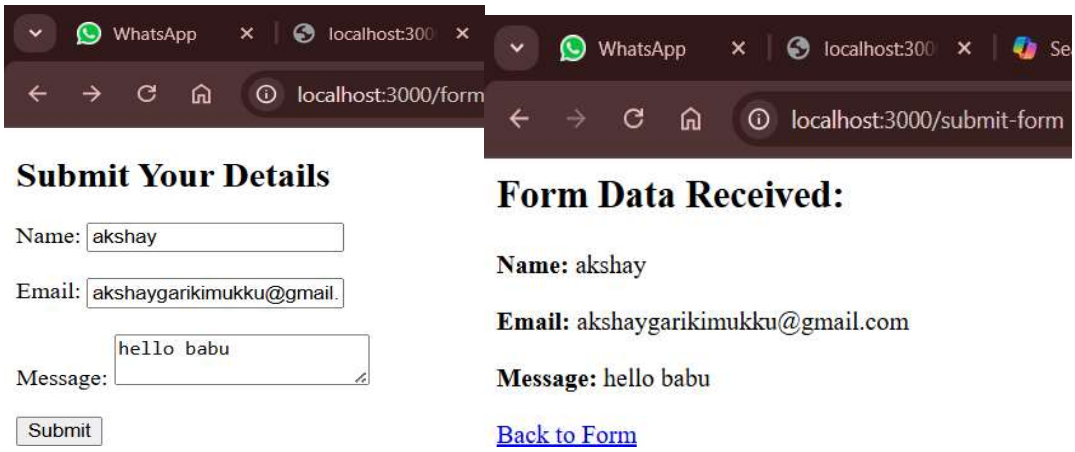
```
const express = require('express');
const app = express();
const PORT = 3000;
app.set('view engine', 'ejs');
app.use(express.urlencoded({ extended: true }));
app.get('/form', (req, res) => {
 res.render('form');
});
app.post('/submit-form', (req, res) => {
 const { name, email, message } = req.body;
 res.send(`
 <h2>Form Data Received:</h2>
 <p><strong>Name:</strong> ${name}</p>
 <p><strong>Email:</strong> ${email}</p>
 <p><strong>Message:</strong> ${message}</p>
 <a href="/form">Back to Form</a>
 `);
});
app.listen(PORT, () => {
```

```
 console.log(`Server running at http://localhost:${PORT}`);
});
```

Views/form.ejs

```
<!DOCTYPE html>
<html>
<head>
  <title>Contact Form</title>
</head>
<body>
  <h1>Contact Us</h1>
  <form action="/submit-form" method="POST">
    <label>Name:</label><br />
    <input type="text" name="name" required /><br /><br />
    <label>Email:</label><br />
    <input type="email" name="email" required /><br /><br />
    <label>Message:</label><br />
    <textarea name="message" required></textarea><br /><br />
    <button type="submit">Submit</button>
  </form>
</body>
</html>
```

Output



6. ExpressJS – Cookies, Sessions, Authentication

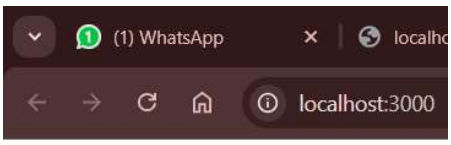
a. Write a program for session management using cookies and sessions.

App.js

```
const express = require('express');
const session = require('express-session');
const cookieParser = require('cookie-parser');
const app = express();
const PORT = 3000;
app.use(cookieParser());
app.use(express.urlencoded({ extended: true }));
app.use(session({
  secret: 'mySecretKey',
  resave: false,
  saveUninitialized: true,
  cookie: { maxAge: 60000 }
}));
app.get('/', (req, res) => {
  if (req.session.username) {
    res.send(`
```

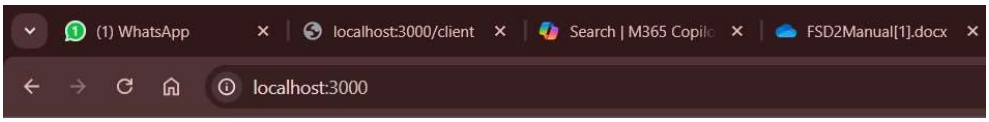
```
 <h2>Welcome, ${req.session.username}!</h2>
 <p>Cookie set: ${req.cookies['connect.sid']}</p>
 <a href="/logout">Logout</a>
 `);
  } else {
     res.send(`
 <h2>Login</h2>
 <form method="POST" action="/login">
 <input type="text" name="username" placeholder="Enter name" required />
 <button type="submit">Login</button>
 </form> `);  }
});
app.post('/login', (req, res) => {
  const { username } = req.body;
  req.session.username = username;
  res.redirect('/');
});
app.get('/logout', (req, res) => {
  res.clearCookie('connect.sid');
  req.session.destroy(err => {
    if (err) {
      return res.send('Error while logging out.');
    }
    res.redirect('/');      });
});
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

Output



Login

akshay Login



Welcome, akshay!

Cookie set: s:CfV5Xz0VyYCMOD9qBgbfn5kYCldfWLpf.Rtubgu3Vi7uRu4xSBc8eecpESXxWduYs0auiXWo79Ac

Logout

10-B

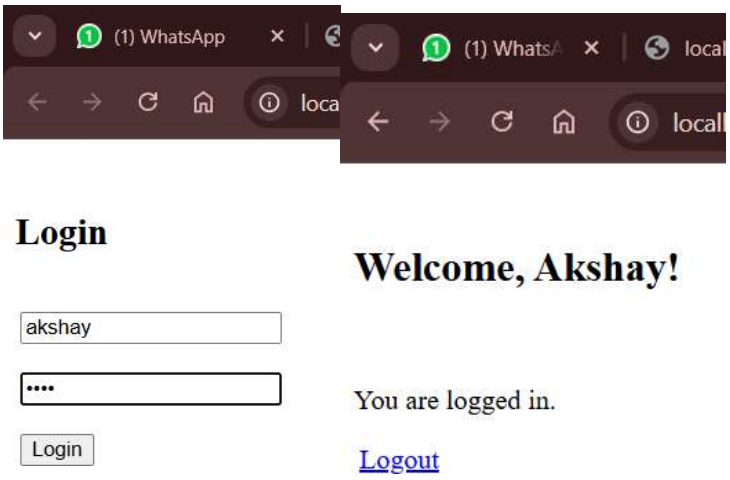
b. Write a program for user authentication

App.js

```
const express = require('express');
const session = require('express-session');
const cookieParser = require('cookie-parser');
const app = express();
const PORT = 3000;
const USER = {
  username: 'Akhil123', password: 'Akhil@123'
};
app.use(cookieParser());
app.use(express.urlencoded({ extended: true }));
app.use(session({
  secret: 'mySecretKey',
```

```
  resave: false,
  saveUninitialized: false,
  cookie: { maxAge: 600000 }
}));
function isAuthenticated(req, res, next) {
  if (req.session.isAuthenticated) {
    next();
  } else {
    res.redirect('/login');
  }
}
app.get('/login', (req, res) => {
  res.send(`
 <h2>Login</h2>
 <form method="POST" action="/login">
 <input type="text" name="username" placeholder="Username" required/><br/><br/>
 <input type="password" name="password" placeholder="Password" required/><br/><br/>
 <button type="submit">Login</button>
 </form> `);
});
app.post('/login', (req, res) => {
  const { username, password } = req.body;
  if (username === USER.username CC password === USER.password) {
    req.session.isAuthenticated = true;
    req.session.username = username;
    res.redirect('/dashboard');
  } else {
    res.send('<p>Invalid credentials. <a href="/login">Try again</a></p>');
```

```
  }
});
app.get('/dashboard', isAuthenticated, (req, res) => {
  res.send(`
 <h2>Welcome, ${req.session.username}!</h2>
 <p>You are logged in.</p>
 <a href="/logout">Logout</a> `);
});
app.get('/logout', (req, res) => {
  req.session.destroy(err => {
    res.clearCookie('connect.sid');
    res.redirect('/login');
  });
});
app.get('/', (req, res) => {
  res.redirect('/login');
});
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

- 

7. ExpressJS – Database, RESTful APIs

a. Write a program to connect MongoDB database using Mongoose and perform CRUD operations.

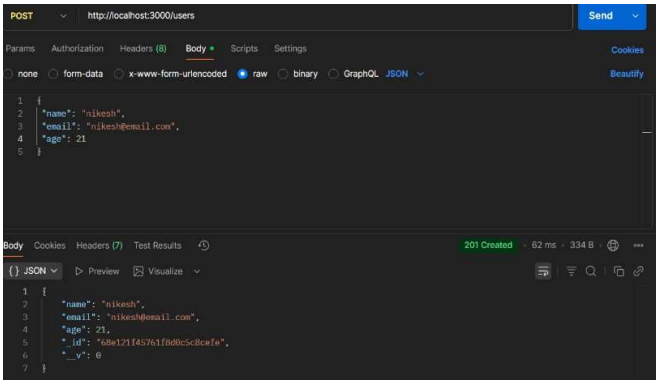
App.js

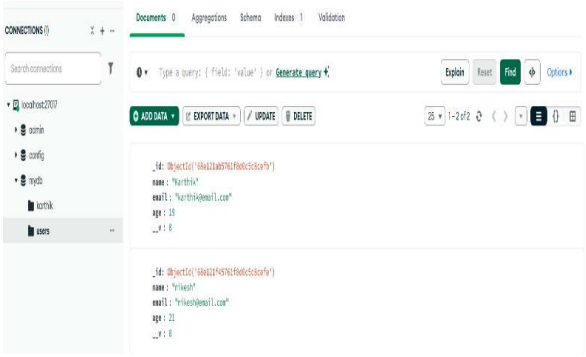
```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const app = express();
const PORT = 3000;
app.use(bodyParser.json());
mongoose.connect('mongodb://127.0.0.1:27017/mydb', {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => console.log("Connected to MongoDB"))
  .catch(err => console.error("MongoDB connection error:", err));
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});
const User = mongoose.model('User', userSchema);
app.post('/users', async (req, res) => {
  try {
    const user = new User(req.body);
    await user.save();
    res.status(201).json(user);
```

```
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});
app.get('/users', async (req, res) => {
  const users = await User.find();
  res.json(users);
});
app.get('/users/:id', async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ message: 'User not found' });
    res.json(user);
  } catch (err) {
    res.status(400).json({ error: 'Invalid ID' });
  }
});
app.put('/users/:id', async (req, res) => {
  try {
    const updatedUser = await User.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    if (!updatedUser) return res.status(404).json({ message: 'User not found' });
    res.json(updatedUser);
  } catch (err) {
    res.status(400).json({ error: err.message });
```

```
  }
});
app.delete('/users/:id', async (req, res) => {
  try {
    const deletedUser = await User.findByIdAndDelete(req.params.id);
    if (!deletedUser) return res.status(404).json({ message: 'User not found' });
    res.json({ message: 'User deleted' });
  } catch (err) {
    res.status(400).json({ error: 'Invalid ID' });
  }
});
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Output [create, read, update, delete]



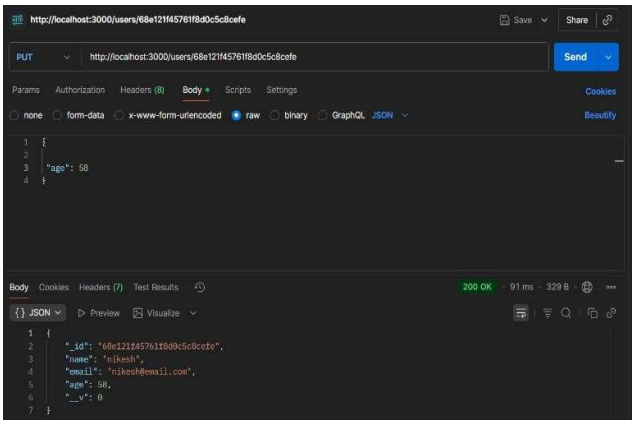


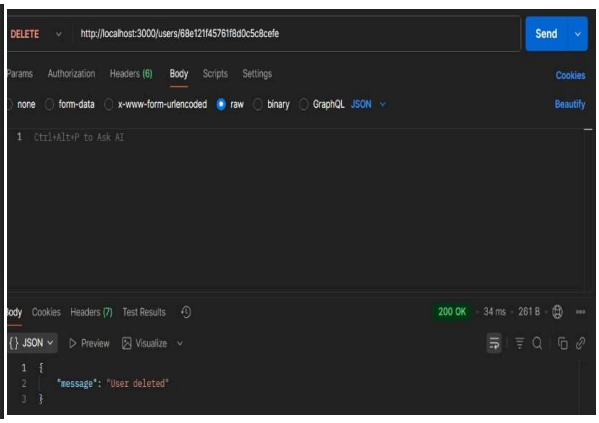
9-A

b. Write a program to develop a single-page application using RESTful APIs.

App.js

```
const express = require('express');
```






```
const bodyParser = require('body-parser');
const path = require('path');
const app = express();
const PORT = 3000;
let items = [];
app.use(bodyParser.json());
app.use(express.static(path.join(__dirname, 'public')));
app.get('/api/items', (req, res) => {
  res.json(items);
});
app.post('/api/items', (req, res) => {
  const { name } = req.body;
  if (!name) return res.status(400).json({ error: 'Name is required' });
  const newItem = { id: Date.now(), name };
```

```
  items.push(newItem);
  res.status(201).json(newItem);
});
app.delete('/api/items/:id', (req, res) => {
  const id = parseInt(req.params.id);
  items = items.filter(item => item.id !== id);
  res.json({ message: 'Item deleted' });
});
app.listen(PORT, () => {
  console.log(`SPA server running at http://localhost:${PORT}`);  });
```

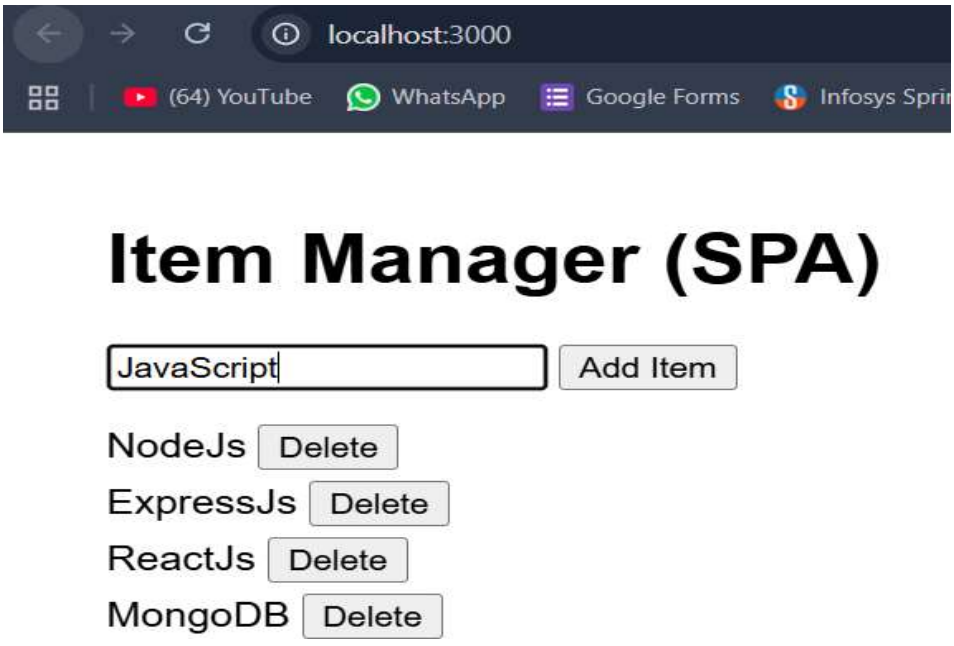
public/index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>SPA with REST API</title>
  <style>
    body {
      font-family: Arial;
      margin: 40px;
    }
    ul {
      list-style: none;
      padding: 0;
    }
    li {margin: 5px 0;}
  </style>
</head>
<body>
```

```
  <h1>Item Manager (SPA)</h1>
  <form id="itemForm">
    <input type="text" id="itemInput" placeholder="Enter item name" required />
    <button type="submit">Add Item</button>
  </form>
  <ul id="itemList"></ul>
  <script>
    const form = document.getElementById('itemForm');
    const input = document.getElementById('itemInput');
    const list = document.getElementById('itemList');
    async function loadItems() {
      const res = await fetch('/api/items');
      const data = await res.json();
      list.innerHTML = '';
      data.forEach(item => {
        const li = document.createElement('li');
        li.innerHTML = `${item.name} <button
onclick="deleteItem(${item.id})">Delete</button>`;
        list.appendChild(li);
      });
    }
    form.addEventListener('submit', async (e) => {
      e.preventDefault();
      const name = input.value.trim();
      if (!name) return;
      await fetch('/api/items', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
```

```
      body: JSON.stringify({ name })
    });
    input.value = '';
    loadItems();
  });
  async function deleteItem(id) {
    await fetch(`/api/items/${id}`, { method: 'DELETE' });
    loadItems();
  }
  loadItems();
 </script>
</body>
</html>
```

Output



9-B

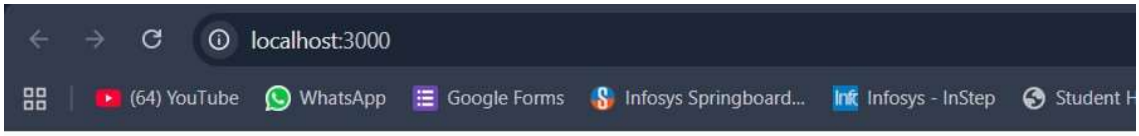
8. ReactJS - Render HTML, JSX, Components - function s Class

a. Write a program to render HTML to a web page.

App.js

```
import React from "react";
function App() {
 return (
  <div>
   <h1>Hello, React!</h1>
   <p>This is rendering plain HTML into a React page.</p>
  </div> );        }
export default App;
```

output



Hello, React!

This is rendering plain HTML into a React page.

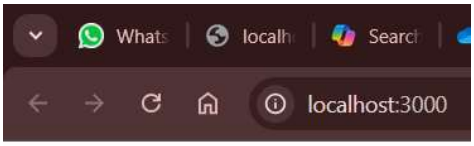
b. Write a program for writing markup with JSX

App.js

```
import React from "react";
import ReactDOM from "react-dom/client";
function App() {
```

```
 const name = "Akhil";
 const currentYear = new Date().getFullYear();
 return (
  <div>
   <h1>Welcome, {name}!</h1>
   <p>The current year is {currentYear}.</p>
  </div> );          }
export default App;
```

Output



Welcome, Akshay!

The current year is 2025.

10-A

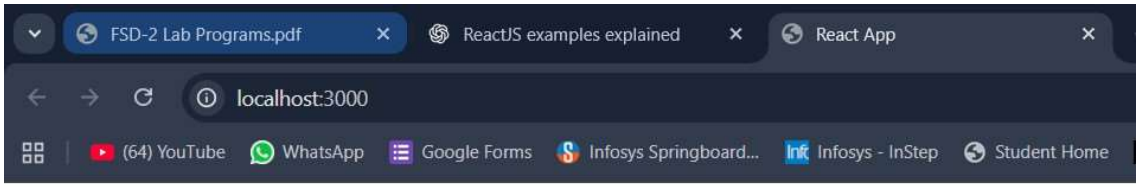
c. Write a program for creating and nesting components (function and class)

App.js

```
import React from "react";
import ReactDOM from "react-dom/client";
function Greeting() {
 return <h2>Hello from Function Component!</h2>;
}
class Welcome extends React.Component {
```

```
 render() {
   return <h2>Hello from Class Component!</h2>;
 }
}
function App() {
 return (
   <div>
    <h1>React Components Example</h1>
    <Greeting />
    <Welcome />
   </div>
 );
}
export default App;
```

Output



React Components Example

Hello from Function Component!

Hello from Class Component!

G. ReactJS – Props and States, Styles, Respond to Events

a. . Write a program to work with props and states.

App.js

```
import React from 'react';
import Counter from './Counter';
import WelcomeMessage from './WelcomeMessage';
function App() {
 return (
   <div style={{ padding: '20px' }}>
    <h1>React Props and State Example</h1>
    <WelcomeMessage user="Akhil" />
    <Counter />
   </div> );
}
export default App;
```

WelcomeMessage.js

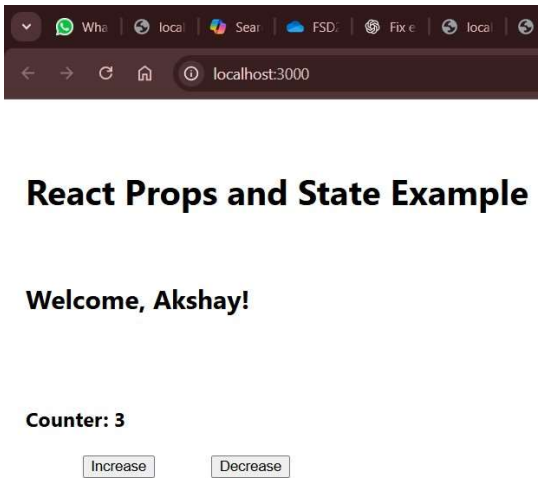
```
import React from 'react';
function WelcomeMessage({ user }) {
 return <h2>Welcome, {user}!</h2>;
}
export  default  WelcomeMessage;
```

Counter.js

```
import React, { Component } from 'react';
class Counter extends Component {
   constructor(props) {
```

```
    super(props);
    this.state = {
      count: 0    };
  }
  increment = () => {
    this.setState(prevState => ({
      count: prevState.count + 1    }));
  };
  decrement = () => {
    this.setState(prevState => ({
      count: prevState.count - 1    }));
  };
  render() {
    return (
      <div>
        <h3>Counter: {this.state.count}</h3>
        <button onClick={this.increment}>Increase</button>
        <button onClick={this.decrement}>Decrease</button>
      </div>  );  }
}
export default Counter;
```

Output



b. Write a program to add styles (CSS C Sass Styling) and display data.

App.js

```
import React from 'react';
import './App.css';
import DataDisplay from './DataDisplay';
function App() {
 return (
  <div className="app-container">
   <h1>React Styling Example</h1>
   <DataDisplay />
  </div> );
}
export default App;
```

DataDisplay.js

```
import React from 'react';
import './DataDisplay.css';
const data = [
  { id: 1, name: 'Nikesh', role: 'Frontend Developer' },
  { id: 2, name: 'Prabhas', role: 'Backend Developer' },
```

```
  { id: 3, name: 'Balaji', role: 'Full Stack Developer' }
];
function DataDisplay() {
  return (
    <div className="data-display">
      {data.map(item => (
        <div  key={item.id}  className="card">
          <h2>{item.name}</h2>
          <p>{item.role}</p>
        </div>
      ))}
    </div>   );
}
export default DataDisplay;
```

app.css

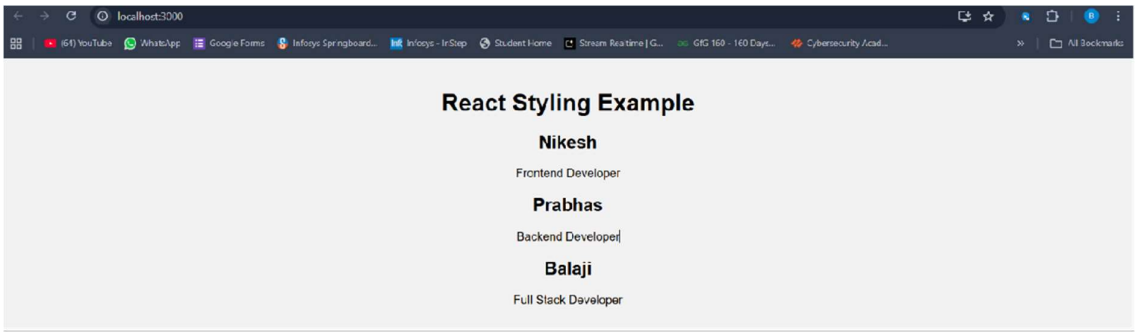
```
.app-container {
 text-align: center;
 background-color: #f2f2f2;
 padding: 20px;
 font-family:  Arial,  sans-serif;
}
```

DataDisplay.css

```
$data-bg: #ffffff;
$card-border: #ccc;
$shadow: rgba(0, 0, 0, 0.1);
.data-display {
  display: flex;
  justify-content:  center;
```

```
  flex-wrap: wrap;
  gap: 20px;
  .card {
    background: $data-bg;
    border: 1px solid $card-border;
    padding: 15px;
    border-radius: 10px;
    box-shadow: 0 4px 6px $shadow;
    width: 200px;
    transition: transform 0.2s ease-in;
    C:hover {
      transform: scale(1.05);
    }
    h2 {
      margin: 0;
      font-size: 20px;
    }
    p {
      color: #666;
    }
  }
}
```

Output



c. Write a program for responding to events.

App.js

```
import React from 'react';
import EventResponse from './EventResponse';
function App() {
 return (
  <div>
   <EventResponse />
  </div>
 );
}
export default App;
```

EventResponse.js

```
import React, { useState } from 'react';
import './styles/EventResponse.css';
function EventResponse() {
  const [bgColor, setBgColor] = useState('#f2f2f2');
  const [name, setName] = useState('');
  const [hoverMessage, setHoverMessage] = useState('');
  const handleClick = () => {
    const randomColor = '#' + Math.floor(Math.random() * 16777215).toString(16);
```

```
    setBgColor(randomColor);
  };
  const handleInputChange = (e) => {
    setName(e.target.value);
  };
  const handleMouseEnter = () => {
    setHoverMessage('Mouse is over the box!');
  };
  const handleMouseLeave = () => {
    setHoverMessage('');
  };
  return (
    <div className="event-container" style={{ backgroundColor: bgColor }}>
      <h2>React Event Handling Example</h2>
      <input type="text"placeholder="Type your name" value={name}
        onChange={handleInputChange} />
      <p>Hello, {name || 'Guest'}!</p>
      <button onClick={handleClick}>Change Background Color</button>
      <div
        className="hover-box"
        onMouseEnter={handleMouseEnter}
        onMouseLeave={handleMouseLeave}
      >
        Hover over me!
      </div>
      <p>{hoverMessage}</p>
    </div>  );
}
```

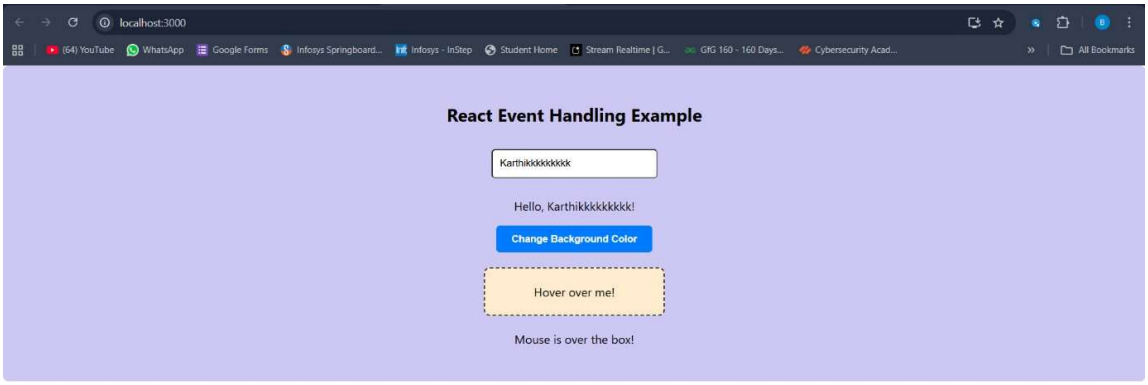
```
export default EventResponse;
```

EventResponse.css

```
.event-container {
  text-align: center;
  padding: 30px;
  border-radius: 8px;
  min-height: 300px;
  transition: background-color 0.3s ease-in-out;
}
input {
  padding: 10px;
  width: 200px;
  margin: 10px;
  border-radius: 5px;
}
button {
  padding: 10px 20px;
  background-color: #007bff;
  border: none;
  color: white;
  font-weight: bold;
  cursor: pointer;
  border-radius: 5px;    }
button:hover {
  background-color: #0056b3;       }
.hover-box {
  margin: 20px auto;
  padding: 20px;
```

```
background-color: #ffebcd;
width: 200px;
border: 2px dashed #333;
cursor: pointer;
border-radius: 8px;      }
```

Output



10. ReactJS – Conditional Rendering, Rendering Lists, React Forms

a. Write a program for conditional rendering

App.js

```
import React from 'react';
import ConditionalRender from './ConditionalRender';
function App() {
 return (
   <div>
    <ConditionalRender />
   </div>    );
}
export default App;
```

ConditionalRender.js

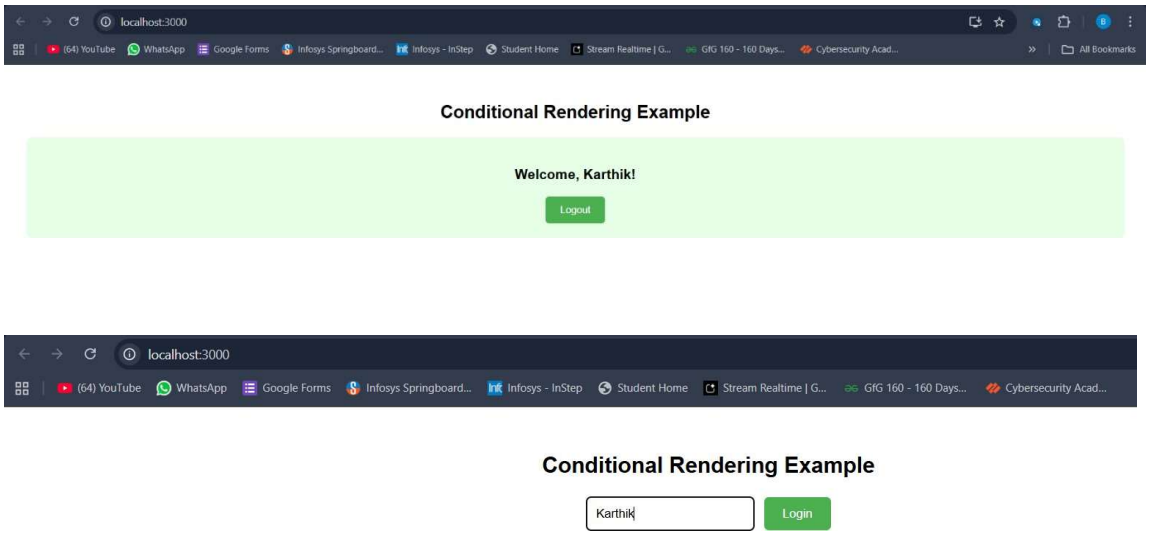
```
import React, { useState } from 'react';
import './styles/ConditionalRender.css';
```

```
function ConditionalRender() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  const [username, setUsername] = useState('');
  const handleLogin = () => {
    if (username.trim() !== '') {
      setIsLoggedIn(true);    }
  };
  const handleLogout = () => {
    setIsLoggedIn(false);
    setUsername('');
  };
  return (
    <div className="container">
      <h2>Conditional Rendering Example</h2>
      {isLoggedIn ? (
        <div className="welcome">
          <h3>Welcome, {username}!</h3>
          <button onClick={handleLogout}>Logout</button>
        </div>    ) : (
        <div className="login-form">
          <input type="text" placeholder="Enter your name" value={username}
            onChange={(e) => setUsername(e.target.value)}  />
          <button onClick={handleLogin}>Login</button>
        </div>)}
    </div>   );
}
export default ConditionalRender;
```

ConditionalRender.css

```
.container {
  text-align: center;
  padding: 30px;
  font-family: sans-serif;
}
.login-form input {
  padding: 10px;
  margin-right: 10px;
  border-radius: 5px;
  border: 1px solid #ccc;
}
button {
  padding: 10px 20px;
  background-color: #4caf50;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}
button:hover {
  background-color: #388e3c;
}
.welcome {
  background-color: #e6ffe6;
  padding: 20px;
  border-radius: 8px;
}
```

Output



b. Write a program for rendering lists.

App.js

```
import React from 'react';
import UserList from './UserList';
function App() {
 return (
  <div>
   <UserList />
  </div>
 );
}
export default App;
```

UserList.js

```
import React from 'react';
import './styles/UserList.css';
const users = [
```

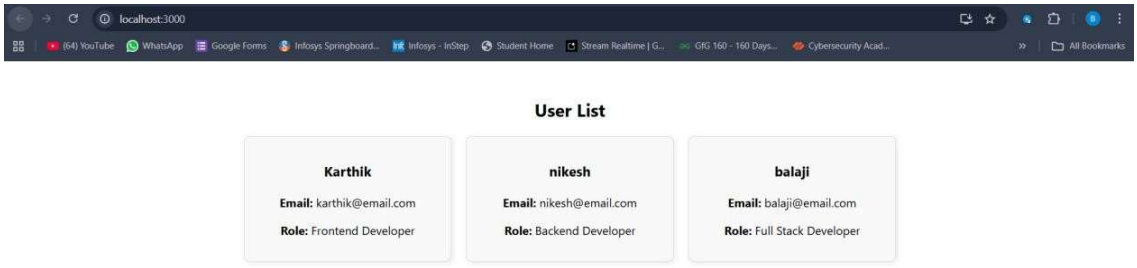
```
  { id: 1, name: 'Karthik', email: 'karthik@email.com', role: 'Frontend Developer' },
  { id: 2, name: 'nikesh', email: 'nikesh@email.com', role: 'Backend Developer' },
  { id: 3, name: 'balaji', email: 'balaji@email.com', role: 'Full Stack Developer' },
];
function UserList() {
  return (
    <div className="user-list-container">
      <h2>User List</h2>
      <ul className="user-list">
        {users.map((user) => (
          <li key={user.id} className="user-card">
            <h3>{user.name}</h3>
            <p><strong>Email:</strong> {user.email}</p>
            <p><strong>Role:</strong> {user.role}</p>
          </li>       ))}
      </ul>
    </div>  );
}
export default UserList;
```

UserList.css

```
.user-list-container {
  padding: 30px;
  text-align: center;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
.user-list {
  list-style: none;
  padding: 0;
```

```
  display: flex;
  justify-content: center;
  flex-wrap: wrap;
  gap: 20px;
}
.user-card {
  background-color: #f9f9f9;
  border: 1px solid #ddd;
  padding: 15px;
  width: 250px;
  border-radius: 8px;
  box-shadow: 2px 2px 8px rgba(0, 0, 0, 0.1);
  transition: transform 0.2s ease-in-out;
}
.user-card:hover {
  transform: scale(1.03);
}
```

Output



c. Write a program for working with different form fields using React Forms

App.js

```
import React from 'react';
```

```
import FormExample from './FormExample';
function App() {
 return (
  <div>
   <FormExample />
  </div>
 );
}
export default App;
```

FormExample.js

```
import React, { useState } from 'react';
import './styles/FormExample.css';
function FormExample() {
  const [formData, setFormData] = useState({
    username: '', email: '',password: '',gender: '',  subscribe: false,  country: 'India',
    about: '' });
  const [submittedData, setSubmittedData] = useState(null);
  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setFormData((prev) => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value
    }));
  };
  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmittedData(formData);
  };
```

```
  return (
    <div className="form-container">
      <h2>React Form Example</h2>
      <form onSubmit={handleSubmit}>
        <label>
          Username:
          <input type="text" name="username" value={formData.username}
            onChange={handleChange} required />
        </label>
        <label>
          Email:
          <input type="email" name="email" value={formData.email}
onChange={handleChange}       required />
        </label>
        <label>
          Password:
          <input type="password" name="password" value={formData.password}
            onChange={handleChange} required />
        </label>
        <div>
          Gender:
          <label><input type="radio" name="gender" value="Male"
onChange={handleChange} />
            Male</label>
          <label><input type="radio" name="gender" value="Female"
onChange={handleChange}
          /> Female</label>
        </div>
        <label>
```

```
            Subscribe to newsletter:
            <input type="checkbox" name="subscribe"checked={formData.subscribe}
              onChange={handleChange} />
          </label>
          <label>
            Country:
            <select name="country" value={formData.country}
onChange={handleChange}>
              <option value="India">India</option>
              <option value="USA">USA</option>
              <option value="UK">UK</option>
            </select>
          </label>
          <label>
            About You:
            <textarea name="about" value={formData.about}
              onChange={handleChange}></textarea>
          </label>
          <button type="submit">Submit</button>
        </form>
        {submittedData CC (
          <div className="result">
            <h3>Submitted Data:</h3>
            <pre>{JSON.stringify(submittedData, null, 2)}</pre>
          </div>
        )}
      </div>   );
}
```

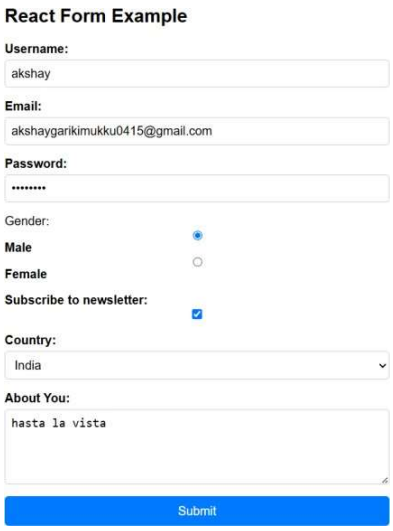
```
export default FormExample;
```

FormExample.css

```
.form-container {
  max-width: 500px;
  margin: auto;
  padding: 20px;
  font-family: Arial, sans-serif;
}
form {
  display: flex;
  flex-direction: column;
  gap: 15px;
}
label {
  display: flex;
  flex-direction: column;
  font-weight: bold;
}
input[type="text"],
input[type="email"],
input[type="password"],
select,
textarea {
  padding: 8px;
  font-size: 16px;
  margin-top: 5px;
  border-radius: 5px;
  border: 1px solid #ccc;
```

```
}
textarea {
  resize: vertical;
  height: 80px;
}
button {
  padding: 10px;
  background-color: #007bff;
  color: white;
  border: none;
  border-radius: 5px;
  font-size: 16px;
  cursor: pointer;
}
button:hover {
  background-color: #0056b3;
}
.result {
  background-color: #f3f3f3;
  margin-top: 20px;
  padding: 15px;
  border-radius: 5px;
}
```

Output



11. ReactJS – React Router, Updating the Screen

a. Write a program for routing to different pages using react router.

App.js

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import Contact from './pages/Contact';
import './styles/App.css';
function App() {
 return (
  <Router>
   <div className="app">
    <nav className="navbar">
     <Link to="/">Home</Link>
     <Link to="/about">About</Link>
```

```
          <Link to="/contact">Contact</Link>
         </nav>
         <div className="page-content">
          <Routes>
           <Route path="/" element={<Home />} />
           <Route path="/about" element={<About />} />
           <Route path="/contact" element={<Contact />} />
          </Routes>
         </div>
        </div>
       </Router>
      );
    }
    export default App;
```

pages/Home.js

```
import React from 'react';
function Home() {
 const homeStyle = {
  textAlign: 'center',
  backgroundColor: '#f0f8ff',
  color: '#333',
  padding: '50px',
  fontFamily: 'Arial, sans-serif',
 };
 const headingStyle = {
  color: '#004aad',
 };
 const paragraphStyle = {
```

```
  fontSize: '18px',
  marginTop: '10px',
 };
 return (
  <div style={homeStyle}>
   <h1 style={headingStyle}>Welcome to SRKR College</h1>
   <p style={paragraphStyle}>
    SRKR College is dedicated to excellence in education and research, shaping future leaders through knowledge and innovation.
   </p>
  </div>
 );
}
export default Home;
```

pages/About.js

```
import React from 'react';
function About() {
 return <h2>This is the About Page.</h2>;
}
export default About;
```

pages/Contact.js

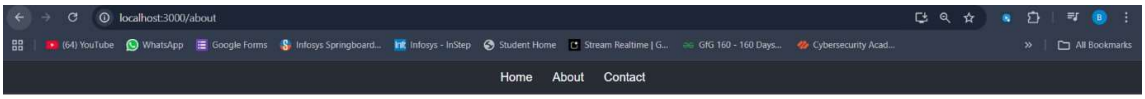
```
import React from 'react';
function Contact() {
 return <h2>Get in touch via the Contact Page.</h2>;
}
export default Contact;
```

styles/App.css

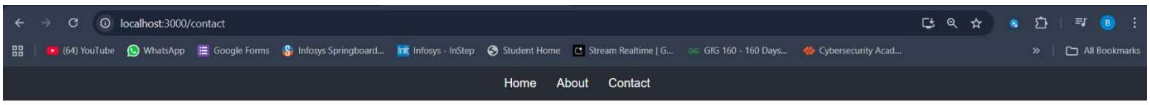
```
.app {
```

```
 font-family: Arial, sans-serif;
 text-align: center;
}
.navbar {
 background-color: #282c34;
 padding: 15px;
}
.navbar a {
 color: white;
 margin: 0 15px;
 text-decoration: none;
 font-size: 18px;
}
.navbar a:hover {
 text-decoration: underline;
}
.page-content {
 padding: 40px;
 font-size: 20px;
}
```

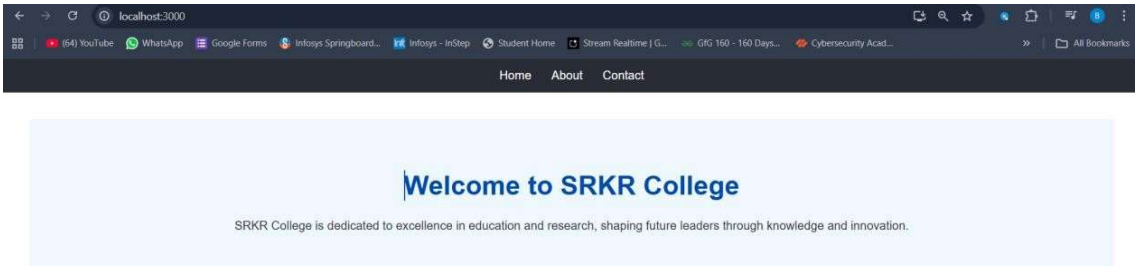
Output



This is the About Page.



Get in touch via the Contact Page.



b. Write a program for updating the screen

LiveClock.js

```
import React, { useState, useEffect } from 'react';
import './styles/LiveClock.css';
function LiveClock() {
  const [time, setTime] = useState(new Date());
  useEffect(() => {
    const interval = setInterval(() => {
      setTime(new Date());
    }, 1000);
    return () => clearInterval(interval);
  }, []);
  return (
    <div className="clock-container">
      <h2>Live Clock</h2>
```

```
      <div className="time-display">{time.toLocaleTimeString()}</div>
    </div>);
}
export default LiveClock;
```

App.js

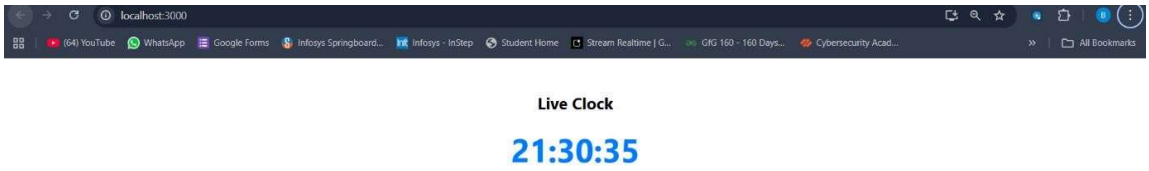
```
import React from 'react';
import LiveClock from './LiveClock';
function App() {
 return (
  <div>
   <LiveClock />
  </div>
 );          }
export default App;
```

styles/LiveClock.css

```
import React, { useState, useEffect } from 'react';
import './styles/LiveClock.css';
function LiveClock() {
  const [time, setTime] = useState(new Date());
  useEffect(() => {
    const interval = setInterval(() => {
      setTime(new Date());
    }, 1000);
    return () => clearInterval(interval);
  }, []);
  return (
    <div className="clock-container">
      <h2>Live Clock</h2>
```

```
        <div className="time-display">{time.toLocaleTimeString()}</div>
      </div>
    );        }
export default LiveClock;
```

Output



12. ReactJS – Hooks, Sharing data between Components

a. Write a program to understand the importance of using hooks.

CounterWithHooks.js

```
import React, { useState, useEffect } from 'react';
import './styles/App.css';
function CounterWithHooks() {
  const [count, setCount] = useState(0);
  // Side effect: update document title when count changes
  useEffect(() => {
    document.title = `You clicked ${count} times`;
```

```
    console.log(`Count updated to ${count}`);
  }, [count]); // runs only when `count` changes
  return (
    <div className="counter-container">
      <h2>React Hooks Example</h2>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>  );
}
export default CounterWithHooks;
```

App.js

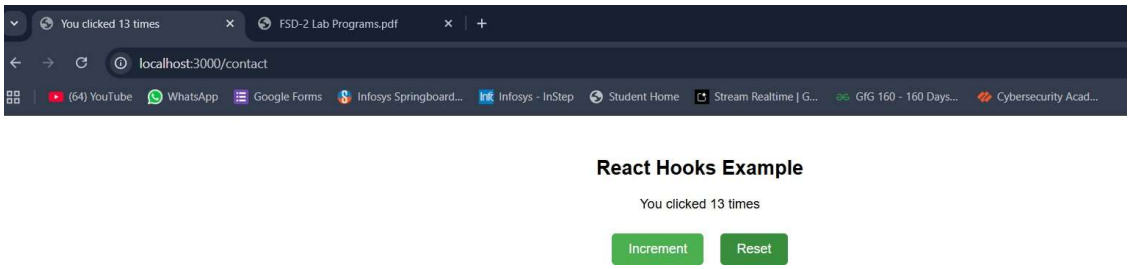
```
import React from 'react';
import CounterWithHooks from './CounterWithHooks';
function App() {
 return (
  <div>
<CounterWithHooks />
  </div> );
}
export default App;
```

styles/App.css

```
.counter-container {
  text-align: center;
  padding: 30px;
}
button {
```

```
  padding: 10px 20px;
  margin: 10px;
  background-color: #4caf50;
  color: white;
  font-size: 16px;
  border-radius: 5px;
  cursor: pointer;
}
button:hover {
  background-color: #388e3c;
}
```

Output



b. Write a program for sharing data between components.

App.js

```
import React, { useState } from 'react';
import NameInput from './components/NameInput';
import NameDisplay from './components/NameDisplay';
import './styles/App.css';
function App() {
 const [name, setName] = useState('');
 return (
  <div className="app-container">
```

```
    <h2>Sharing Data Between Components</h2>
    <NameInput name={name} setName={setName} />
    <NameDisplay name={name} />
   </div> );
}
export default App;
```

components/NameInput.js

```
import React from 'react';
function NameInput({ name, setName }) {
  return (
    <div>
      <input type="text" placeholder="Enter your name" value={name}
        onChange={(e) => setName(e.target.value)} />
    </div>  );
}
export default NameInput;
```

components/NameDisplay.js

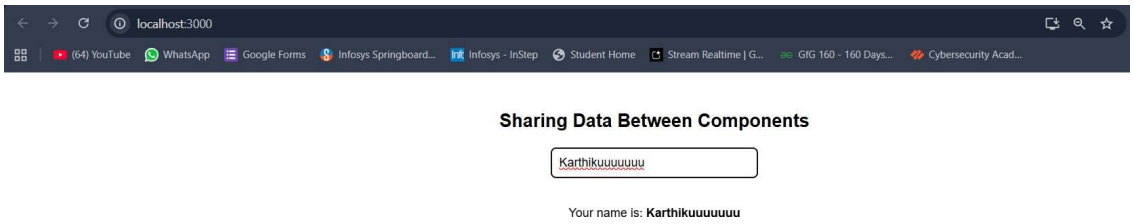
```
import React from 'react'; function NameDisplay({ name }) {
  return (
    <div>
      <p>Your name is: <strong>{name || 'Not provided'}</strong></p>
    </div> );
}
export default NameDisplay;
```

styles/app.css

```
.app-container {
 text-align: center;
```

```
 padding: 30px;
 font-family: Arial, sans-serif;
}
input {
 padding: 10px;
 width: 250px;
 font-size: 16px;
 border-radius: 6px;
 border: 1px solid #ccc;
 margin-bottom: 20px;
}
```

Output



13. ReactJS Applications – To-do list and Quiz

Design a to-do list application

App.js

```
import React, { useState } from 'react';
import Todo from './Todo';
import './styles/Todo.css';
function App() {
 const [todos, setTodos] = useState([]);
```

```
const [input, setInput] = useState('');
const handleAdd = () => {
 if (input.trim() !== '') {
  const newTodo = {
  id: Date.now(),
   text: input,
   completed: false,
  };
  setTodos([...todos, newTodo]);
  setInput('');
 }
};
const handleToggle = (id) => {
 setTodos(
  todos.map((todo) =>
   todo.id === id ? { ...todo, completed: !todo.completed } : todo
  )
 );
};
const handleDelete = (id) => {
 setTodos(todos.filter((todo) => todo.id !== id));
};
return (
 <div className="todo-app">
  <h2>To-Do List</h2>
  <div className="todo-input">
   <input type="text" placeholder="Add a new task..."value={input}
    onChange={(e) => setInput(e.target.value)} />
```

```
      <button onClick={handleAdd}>Add</button>
    </div>
    <ul className="todo-list">
     {todos.map((todo) => (
      <Todo key={todo.id}
        todo={todo}  onToggle={handleToggle}  onDelete={handleDelete}    />
     ))}
    </ul>
  </div> );
}
export default App;
```

Todo.js

```
import React from 'react';
function Todo({ todo, onToggle, onDelete }) {
  return (
    <li className={`todo-item ${todo.completed ? 'completed' : ''}`}>
      <span onClick={() => onToggle(todo.id)}>{todo.text}</span>
      <button className="delete" onClick={() => onDelete(todo.id)}>+</button>
    </li>   );
}
export default Todo;
```

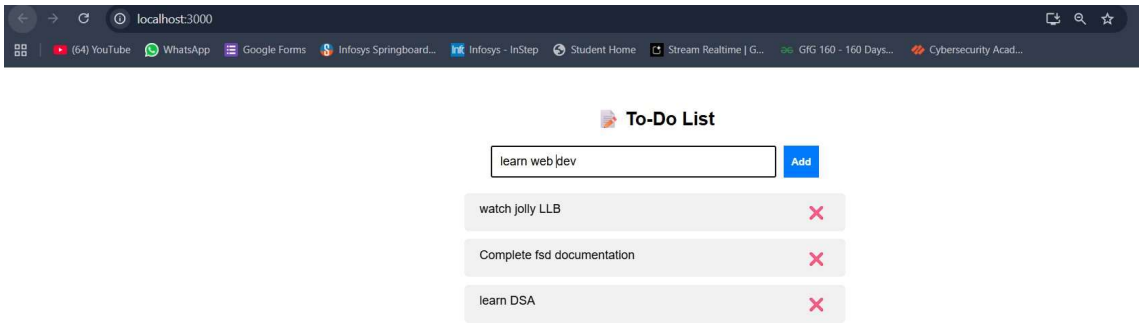
App.css

```
.todo-app {
  max-width: 500px;
  margin: auto;
  padding: 30px;
  font-family: Arial, sans-serif;
```

```
    text-align: center;     }
.todo-input {
    display: flex;
    justify-content: center;
    margin-bottom: 20px;  }
.todo-input input {
    width: 70%;
    padding: 10px;
    font-size: 16px;   }
.todo-input button {
    padding: 10px;
    margin-left: 10px;
    background-color: #007bff;
    color: white;
    border: none;
    font-weight: bold;
    cursor: pointer; }
.todo-list {
    list-style: none;
    padding: 0;   }
.todo-item {
    background-color: #f1f1f1;
    margin: 10px 0;
    padding: 12px 20px;
    display: flex;
    justify-content: space-between;
    border-radius: 6px;
    transition: 0.3s ease;
```

```
    cursor: pointer;  }
.todo-item.completed span {
    text-decoration: line-through;
    color: gray;  }
.todo-item .delete {
    background: transparent;
    border: none;
    color: red;
    font-size: 18px;
    cursor: pointer;
}
```

Output



14. MongoDB – Installation, Configuration, CRUD operations

a. Install MongoDB and configure MongoDB Atlas

Step 1: Download MongoDB

Visit: https://www.mongodb.com/try/download/community

Choose your OS (Windows, macOS, Linux)

Select MSI (for Windows) or TGZ/ZIP (for Linux/macOS)

Download and install

On Windows, install with the MongoDB Compass GUI and ensure MongoDB Server and MongoDB Shell (mongosh) are selected.

Step 2: Verify Installation

Open terminal or command prompt and run: mongosh

You should see a prompt like: test>

This confirms MongoDB is installed and running.

By default, MongoDB runs on port 27017.

Step 3: Start MongoDB (if not running)

Windows: MongoDB runs as a service after installation.

macOS/Linux: Run it manually: brew services start mongodb-community

mongod --dbpath /your/data/folder

Configure MongoDB Atlas (Cloud)

Step 1: Create Account

1. Visit https://www.mongodb.com/cloud/atlas
2. Sign up (free tier is available)

Step 2: Create a Cluster

1. Click "Build a Cluster"
2. Choose the free tier (M0 Shared Cluster)
3. Select a cloud provider (AWS, GCP, Azure)
4. Select a region close to you
5. Name your cluster (e.g., myCluster)
6. Click Create Cluster

Step 3: Create a Database User

1. Go to "Database Access"
2. Click "Add New Database User"
3. Set username and password
4. Choose "Read and write to any database"
5. Click Add User

Step 4: Set Network Access (Whitelist IP)

1. Go to "Network Access"
2. Click "Add IP Address"
3. Choose:0.0.0.0 — to allow access from anywhere (for development)

   or enter your public IP for security
4. Click Confirm

Step 5: Connect to Your Cluster

1. Go to "Clusters" > Click "Connect"
2. Choose: Connect your application
3. Copy the connection string, e.g.:

   mongodb+srv://<username>:<password>@cluster0.mongodb.net/?retryWrites=trueCw=majority

   Replace <username> and <password> with your DB credentials.

b. Write MongoDB queries to perform CRUD operations on document using insert(), find(), update(), remove()

Select Database

- use collegeDB

INSERT (CREATE)

- db.students.insertOne({ name: "Karthik",age: 19,department: "Computer Science",grade: "A" });

Output:

{ acknowledged: true, insertedId: ObjectId("...") }

- db.students.insertMany([

  { name: "Sameer", age: 20, department: "Mathematics", grade: "B" },

  { name: "Vivek", age: 21, department: "Physics", grade: "A" }]);

Output:

{ acknowledged: true, insertedIds: [ObjectId("..."), ObjectId("...")] }

FIND (READ)

- db.students.find();

Output:

{ "_id" : ObjectId("68e23e3f7256aeb539f2baf0"), "name" : "Karthik", "age" : 19, "department" : "Computer Science", "grade" : "A" }

{ "_id" : ObjectId("68e23e3f7256aeb539f2baf1"), "name" : "Sameer", "age" : 20, "department" : "Mathematics", "grade" : "B" }

{ "_id" : ObjectId("68e238b77256aeb539f2bae "), "name" : "Vivek", "age" : 21, "department" : "Physics", "grade" :"A" }

- db.students.find({ grade: "A" });

Output:

{ "_id" : ObjectId("68e23e3f7256aeb539f2baf0"), "name" : "Karthik", "age" : 19, "department" : "Computer Science", "grade" : "A" }

{ "_id" : ObjectId("68e238b77256aeb539f2baef"), "name" : "Vivek", "age" : 21, "department" : "Physics", "grade" : "A" }

- db.students.findOne({ name: "Sameer" });

Output:

{ "_id" : ObjectId("68e23e3f7256aeb539f2baf1"), "name" : "Sameer", "age" : 20, "department" : "Mathematics", "grade" : "B" }

- db.students.find({}, { name: 1, age: 1, _id: 0 });

Output:

{ "name" : "Karthik", "age" : 19 }

{ "name" : "Sameer", "age" : 20 }

{ "name" : "Vivek", "age" : 21 }

UPDATE (MODIFY)

- db.students.updateOne({ name: "Vivek" }, { $set: { age: 23, grade: "A+" } });

Output:

{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }

- db.students.updateMany({ grade: "B" }, { $set: { grade: "B+" } });

Output:

{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }

DELETE (REMOVE)

- db.students.deleteOne({ name: "Karthik" });

Output:

{ acknowledged: true, deletedCount: 1 }

- db.students.deleteMany({ grade: "B+" });

Output:

{ acknowledged: true, deletedCount: 1 }

- db.students.find();

Output:

{ "_id" : ObjectId("68e238b77256aeb539f2baef"), "name" : "Vivek", "age" : 23, "department" : "Physics", "grade" : "A+" }

15. MongoDB – Databases, Collections and Records

a. Write MongoDB queries to Create and drop databases and collections.

Create a Database

- use mydb

Output: switched to db mydb

Create a Collection

- db.createCollection("students")

Output: { ok: 1 }

Show All Databases

- show dbs

Output: admin 40.00 KiB
collegeDB 72.00 KiB
config 72.00 KiB
mydb 80.00 KiB

Show All Collections in Current Database

- show collections

Output: karthik

students

users

Drop a Collection

- db.students.drop()

Output: true

Drop a Database

- use mydb // already on mydb
- db.dropDatabase()

Output: { ok: 1, dropped: 'mydb' }

b. Write MongoDB queries to work with records using find(), limit(), sort(), createIndex(), aggregate().

Create and insert data in database

- use collegeDB
- switched to db collegeDB
- db.students.insertMany([

{ name: "Nikesh", age: 20, grade: "A", department: "CS" },

{ name: "Balaji", age: 21, grade: "B", department: "Math" },

{ name: "Prabhs", age: 20, grade: "C", department: "CS" },

{ name: "Sameer", age: 20, grade: "C", department: "Physics" },

{ name: "Vivek", age: 19, grade: "B", department: "CS" }]);

find() - Retrieve documents

- db.students.find()

Output: //All students records will appear

- db.students.find({ grade: "A" })

Output: {

_id: ObjectId('68e2714c3253243ca86fcfeb'),

name: 'Nikesh',

age: 20,

grade: 'A',

department: 'CS' }

limit() - Limit the number of results

- db.students.find().limit(1)

Output: {

_id: ObjectId('68e23e3f7256aeb539f2baf0'),

name: 'Karthik',

age: 19,

department: 'Computer Science',

grade: 'A' }

sort() – Sort the results

- db.students.find().sort({ age: 1 })

Output: // vivek , Nikesh , prbhas, sameer, Balaji

createIndex() – Index for performance

- db.students.createIndex({ name: 1 })

Output: name_1

aggregate() – Perform aggregation operations

- db.students.aggregate([{ $group: { _id: "$department", total: { $sum: 1 } } }])

Output: {_id: 'Math', total: 1}

{_id: 'CS', total: 3}

{_id: 'Physics',total: 1}

- db.students.aggregate([{ $group: { _id: "$grade", avgAge: { $avg: "$age" } } }])

Output: { _id: 'C', avgAge: 20}

{ _id: 'A', avgAge: 20}

{_id: 'B',avgAge: 20}

16. Augmented Programs 2

a. Design a to-do list application using NodeJS and ExpressJS

App.js

```
const express = require('express');
const bodyParser = require('body-parser');
const app = express();
const PORT = 3000;
app.use(bodyParser.urlencoded({ extended: true }));
app.use(express.static('public'));
app.set('view engine', 'ejs');
let tasks = [];
app.get('/', (req, res) => {
  res.render('index', { tasks });
});
app.post('/add', (req, res) => {
  const task = req.body.task;
  if (task) tasks.push(task);
  res.redirect('/');
});
app.post('/delete', (req, res) => {
  const index = parseInt(req.body.index);
  if (!isNaN(index)) tasks.splice(index, 1);
  res.redirect('/');
});
app.listen(PORT, () => {
  console.log(`To-Do app running at http://localhost:${PORT}`);
});
```

Views/index.ejs

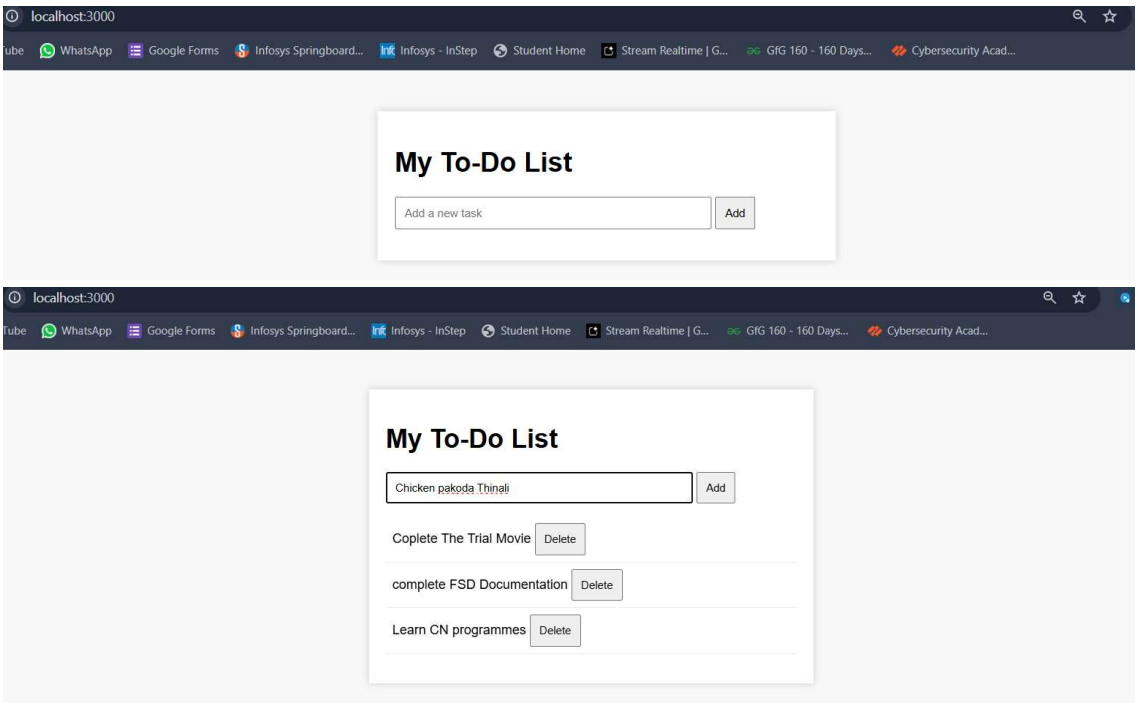
```
<!DOCTYPE html>
<html>
<head>
  <title>To-Do List</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <div class="container">
    <h1>My To-Do List</h1>
    <form action="/add" method="POST">
      <input type="text" name="task" placeholder="Add a new task" required />
      <button type="submit">Add</button>
    </form>
    <ul>
      <% tasks.forEach((task, index)=> { %>
        <li>
          <%= task %>
            <form action="/delete" method="POST" style="display:inline;">
              <input type="hidden" name="index" value="<%= index %>" />
              <button type="submit">Delete</button>
            </form>
        </li>
        <% }) %>
    </ul>
  </div>
</body>
</html>
```

Public/style.css

```
body {
 font-family: Arial;
 background-color: #f9f9f9;
 padding: 40px;
}
.container {
 max-width: 500px;
 margin: auto;
 background: white;
 padding: 20px;
 box-shadow: 0 0 10px #ccc;
}
form input {
 padding: 10px;
 width: 70%;
}
form button {
 padding: 10px;
}
ul {
 list-style-type: none;
 padding: 0;
}
li {
 padding: 8px;
 border-bottom: 1px solid #eee;
}
```

```
li form {
 display: inline;
}
```

Output



b. Design a Quiz app using ReactJS.

App.js

```
import React, { useState } from 'react';
import quizData from './data';
import Question from './components/Question';
import Result from './components/Result';
import './App.css';
function App() {
 const [currentQ, setCurrentQ] = useState(0);
 const [score, setScore] = useState(0);
 const [isFinished, setIsFinished] = useState(false);
```

```
 const handleAnswer = (selected) => {
  if (selected === quizData[currentQ].answer) {
   setScore(score + 1);
  }
  const nextQ = currentQ + 1;
  if (nextQ < quizData.length) {
  setCurrentQ(nextQ);
  } else {
   setIsFinished(true); }
 };
 return (
  <div style={{ maxWidth: "600px", margin: "auto", padding: "20px", fontFamily:
"Arial" }}>
    <h1>Quiz App</h1>
    {isFinished ? (
     <Result score={score} total={quizData.length} />
    ) : (
     <Question data={quizData[currentQ]} index={currentQ}
onAnswer={handleAnswer}/>
    )}
  </div> );  }
export default App;
```

src/Question.js

```
import React from 'react';
const Question = ({ data, index, onAnswer }) => {
  return (
    <div>
      <h2>Question {index + 1}</h2>
```

```
        <p>{data.question}</p>
        {data.options.map((option, idx) => (
          <button  key={idx} onClick={() => onAnswer(option)}
            style={{ display: 'block', margin: '10px 0', padding: '10px' }} >
            {option}
          </button>
        ))}
      </div> );  };
export default Question;
```

src/Result.js

```
import React from 'react';
const Result = ({ score, total }) => {
  return (
    <div>
      <h2>Quiz Complete!</h2>
      <p>Your Score: {score} / {total}</p>
    </div>);
};
export default Result;
```

App.css

```
button {
 background-color: #007bff;
 color: white;
 border: none;
 width: 100%;
 font-size: 16px;
 cursor: pointer;
}
```

```
button:hover {
 background-color: #0056b3;
}
```

Output



